

Available online at www.sciencedirect.com

ScienceDirect

journal homepage: www.elsevier.com/locate/coseComputers
&
Security

A Formal Approach to Network Segmentation

Neerja Mhaskar^{a,*}, Mohammed Alabbad^{a,b}, Ridha Khedri^a^aDept. of Computing and Software McMaster University, Hamilton, Ontario, Canada^bCommunications and Information Technology Research Institute King Abdulaziz City for Science and Technology (KACST), Riyadh, 11442, Saudi Arabia

ARTICLE INFO

Article history:

Received 28 April 2020

Revised 23 November 2020

Accepted 22 December 2020

Available online 5 January 2021

Keywords:

Computer network security

Network architecture

Network segmentation

Layered protection

Defence in depth

Product family algebra

Guarded commands

Software defined networks

ABSTRACT

Network segmentation or compartmentalization, and layered protection are two strategies that are critical in building a secure network. In the literature, layered protection has been formalized and termed as the Defence in Depth (DD) strategy. However, network segmentation has been described vaguely, and without any formal approach, thus making the secure design of large networks unwieldy. In this paper, we formally define network segmentation using a formalism based on product family algebra and guarded commands. Then we propose two algorithms that take a set of resources and their access control policies as input and output a robust network topology and the policies of its firewalls. The firewall policies are computed based on the network segmentation formalism and are strategically placed in the network to achieve DD. Further, we use the proposed algorithms to build Software Defined Networks (SDN) and discuss its use in dynamic networks and Internet of Things.

© 2021 Elsevier Ltd. All rights reserved.

1. Introduction

Firewalls control traffic and separate networks with different security requirements. They are used by organizations to protect their network resources from unauthorized access. They inspect network traffic (packets) and perform actions on it based on the rules, and the actions specified therein in the policies (Cheswick and Bellovin, 1994; R.Vacca and Ellis, 2005). An organization could have up to hundreds of firewalls scattered around its network (R.Vacca and Ellis, 2005), and therefore the placement of firewalls and their policies play a key role in their effectiveness. In practice network administrators decide the placement and configuration of firewalls based on network security design principles and best practices. However, this approach is prone to error, especially for large net-

works. Therefore, a formal approach is needed to achieve and guarantee a secure network design.

Two strategies are followed in network design: *layered protection* (Stawowski, 2007), and *compartmentalization* (Stawowski, 2009) or *segmentation* (May et al., 2006). The principle of *layered protection* (Stawowski, 2007) states that the security of a network should not rely on one single access control point but multiple layers complementing each other to provide maximum security. This strategy is formalized by the Defence in Depth (DD) strategy formally defined in Khedri et al. (2017) and discussed in Khedri et al. (2017); Lippmann et al. (2006); Rubel et al. (2005); U.S. Department of Homeland Security (2016). Therefore a guaranteed robust strategy exists to achieve DD in a network from an access control perspective.

Network segmentation states that resources of different security levels and requirements should be placed in differ-

* Corresponding author.

E-mail address: pophlin@mcmaster.ca (N. Mhaskar).<https://doi.org/10.1016/j.cose.2020.102162>

0167-4048/© 2021 Elsevier Ltd. All rights reserved.

ent zones protected by firewalls implementing appropriate policies. Network segmentation is vital in the design of networks as it implements the separation of concerns principle by putting resources with “similar” policies under the same firewall. The concept of segmentation has so far been discussed in an intuitive way (Wagner et al., 2016). For instance, the criteria used for segmenting resources in a network are articulated vaguely, as we find in Stawowski (2007) wordings such as “Information Technology (IT) system resources of different sensitivity levels (i.e., different risk tolerance values and threat susceptibility) should be located in different security zones.” Similar wording can be found in Fægri and Hallstein (2006). While these intuitive criteria for segmentation can be used to design smaller networks, their use for designing a large network is impractical.

It is now a standard policy in network design to create a special subnetwork called Demilitarized Zone (DMZ) between the outer network and the internal network (Cheswick and Bellovin, 1994; R.Vacca and Ellis, 2005; Scarfone and Hoffman, 2009). DMZ usually houses public facing services and systems with similar access control requirements, thereby protecting internal networks from outside access. It is clear that DMZ is a manifestation of the strategies of DD and network segmentation. The rest of the network ought to be segmented in a similar systematic way. In this paper, we explore how to systematically segment a network to enhance its security using a formal approach.

1.1. Related work and motivation

It is evident that network segmentation mitigates security risks (Google Inc., 2012; National Security Agency (NSA), 2013). However, many network segmentation architectures exist even for a small network and these possibilities grow exponentially with the increase in the number of resources in the network. Due to the lack of a formal approach to segment resources to achieve the best network segmentation, this task is left to the discretion of the network administrators. Furthermore, research results that address this problem are very limited with only a handful of papers by Wagner et al. (2017, 2019, 2016).

In Wagner et al. (2017) Wagner et al. develop a Markov chain model to evaluate a given network segmentation architecture which they claim to be cost effective. The advantage of this approach is that security practitioners can use this model to examine multiple candidate architectures and then select the most effective one. However, this solution only evaluates already designed segmentation architectures and does not build them. Further, in Wagner et al. (2016) they present a semi-automated approach to segment resources based on risk evaluation that uses heuristics to reach the near best or in some cases the best segmentation. However, the high cost of this solution makes it impractical for wide-scale use. Finally, in Wagner et al. (2019) they present network segmentation architectures optimized for multiple objectives. Their solution is fully automated and considers the problem of optimizing multiple objectives of security, cost, and mission performance. In contrast, our solution optimizes security in a network, is fully automated and generates one of the best possible segmentations. Furthermore, our solution can be easily scaled to build

enormous networks and is also applicable to Software Defined Networks (SDN) and Internet of Things (IoT) architectures (see Section 7).

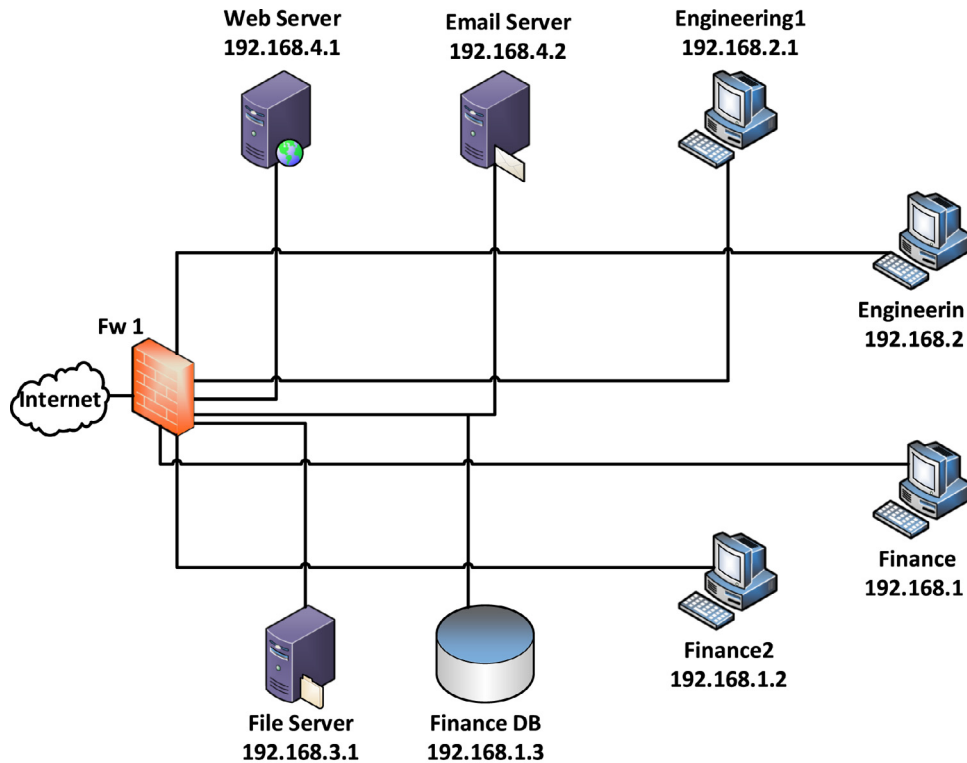
Other solutions exist that aim to secure existing networks. For example, Adão et al. (2016) proposes an algorithm to determine and distribute packet filtering functionality to nodes within a given network. The filtering functions are derived from given security goals for a given network. The work in Adão et al. (2016) is motivated by defining the behavioural security specification in a network. Further, in Rahman and Al-Shaer (2013) the authors propose an automated framework for generating network security configurations and physical placements of security devices. Given a network topology, its security requirements and its business constraints as inputs, the framework formulates the security design synthesis problem and solves it using the Satisfiability Modulo Theories (SMT) (de Moura and Bjørner, 2009). This framework also supports placing security devices in an existing network. In contrast to Adão et al. (2016); Rahman and Al-Shaer (2013), we focus on building a network topology that achieves security strengths by design. The approach presented in Adão et al. (2016); Rahman and Al-Shaer (2013) can be used on the network topology built from our approach to further enhance network security by ensuring that security goals/constraints are met.

To summarize, although many approaches and formalisms exist to build a secure network and harden security in existing networks, a formal approach to achieve network segmentation does not exist. Therefore, the notion of ‘similarity’ or ‘different sensitivity levels’ while grouping resources has not been precisely articulated, resulting in the inability to automate the derivation of the best solution for a network segmentation and prove its correctness. Motivated by these issues, in our paper we present a formal approach to network segmentation, and later use it to build a robust secure network which provides maximum access-protection to its resources.

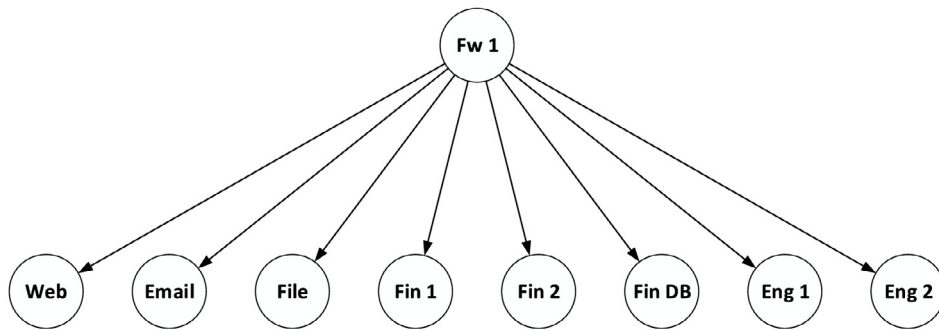
The outline of the paper is as follows. In Section 2, we provide an illustrative example. In Sections 3, we give the mathematical background of the formalism used. In Section 4, we summarize the Strict Defence in Depth (SDD) strategy and present theoretical results needed for the rest of the paper. In Section 5, we formally define segmentation. In Section 6, we present two algorithms to build a guaranteed robust network; the Exponential Robust Network and Segmentation Algorithm (Exp-RNS) and the Robust Network and Segmentation Algorithm (RNS). In Section 7, we discuss the use of the RNS algorithm in an SDN architecture. In Section 8, we discuss few other applications of the RNS algorithm. In Section 9, we discuss the use of the RNS algorithm in different scenarios. Finally, in Section 10, we conclude and discuss avenues for future work.

2. Illustrative example

In this section, we present an example for which manual implementation of the network segmentation principle is straightforward. The example is used for illustration through-



(a) A topological representation of network (1)



(b) A graph representation of network (1)

Fig. 1 – Network Structure (1).

out the paper due to its simplicity in presenting the concepts introduced. Consider an organization with different departments, in which resources are connected in a network. We consider the following small subset of resources: a *Web server*, an *Email server*, two workstations belonging to the Engineering department (i.e., *Engineering1* and *Engineering2*), two workstations and a database belonging to the Finance department (i.e., *Finance1*, *Finance2*, and *Finance DB*), and an internal *File server* available to all departments. A topology of a network for the organization is shown in Fig. 1.

The above resources have different security requirements governing their access. All of them allow an established or a

related connection and drop every invalid packet. The *Web server* and *Email server* allow every HTTP (TCP port 80) and SMTP (TCP port 25) connection from any source, any access from all internal resources, and drop everything else. The *File server* rejects access requests made by the *Web server* and *Email server*, allows all internal resources, and drops every other access request. The resources belonging to the Engineering and Finance departments have stricter policies, which allow access only by users and resources within their respective department, reject all requests made by any other internal resource, and drops every other access request. These security requirements are translated into policies in the language of

```

1 -A INPUT -m state --state RELATED,ESTABLISHED -j ACCEPT
2 -A INPUT -m state --state INVALID -j DROP
3 -A INPUT -p tcp -m state --state NEW -m tcp --dport 80 -j
  ACCEPT
4 -A INPUT -p tcp -m state --state NEW -m tcp --dport 25 -j
  ACCEPT
5 -A INPUT -s 192.168.1.0/24 -j ACCEPT
6 -A INPUT -s 192.168.2.0/24 -j ACCEPT
7 -A INPUT -s 192.168.3.0/24 -j ACCEPT
8 -A INPUT -s 192.168.4.0/24 -j ACCEPT
9 -A INPUT -j DROP

```

Fig. 2 – Web server and Email server policy.

```

1 -A INPUT -m state --state RELATED,ESTABLISHED -j ACCEPT
2 -A INPUT -m state --state INVALID -j DROP
3 -A INPUT -s 192.168.1.0/24 -j ACCEPT
4 -A INPUT -s 192.168.2.0/24 -j ACCEPT
5 -A INPUT -s 192.168.3.0/24 -j ACCEPT
6 -A INPUT -s 192.168.4.0/24 -j REJECT
7 -A INPUT -j DROP

```

Fig. 3 – File server policy.

```

1 -A INPUT -m state --state RELATED,ESTABLISHED -j ACCEPT
2 -A INPUT -m state --state INVALID -j DROP
3 -A INPUT -s 192.168.1.0/24 -j REJECT
4 -A INPUT -s 192.168.2.0/24 -j ACCEPT
5 -A INPUT -s 192.168.3.0/24 -j REJECT
6 -A INPUT -s 192.168.4.0/24 -j REJECT
7 -A INPUT -j DROP

```

Fig. 4 – Engineering workstations policy.

```

1 -A INPUT -m state --state RELATED,ESTABLISHED -j ACCEPT
2 -A INPUT -m state --state INVALID -j DROP
3 -A INPUT -s 192.168.1.0/24 -j ACCEPT
4 -A INPUT -s 192.168.2.0/24 -j REJECT
5 -A INPUT -s 192.168.3.0/24 -j REJECT
6 -A INPUT -s 192.168.4.0/24 -j REJECT
7 -A INPUT -j DROP

```

Fig. 5 – Finance workstations and database policy.

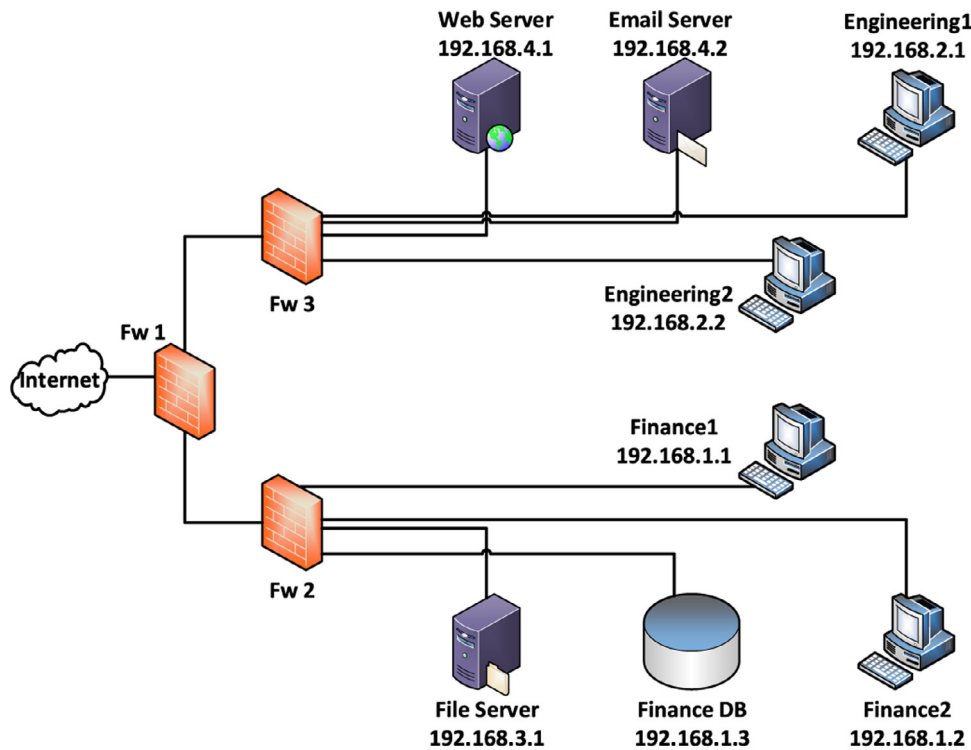
*iptables*¹ and are given in Figs. 2, 3, 4, and 5. Throughout the paper, we use this setup to demonstrate the concepts discussed or introduced.

Observe that the policies in Figs. 2, 3, 4, and 5 have three actions: ACCEPT, REJECT, and DROP. Although firewalls typically perform two actions; one to let the network traffic pass and the other to block it, some firewalls perform more actions.

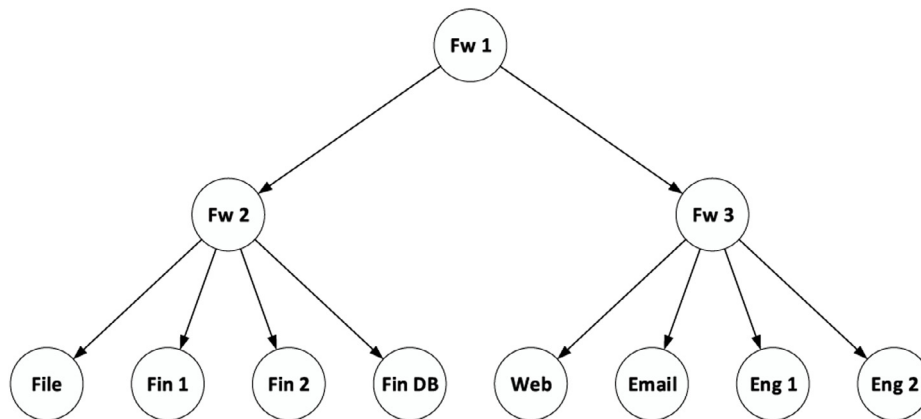
¹ *iptables* is a command line utility program for configuring Linux kernel firewall

For example, in *iptables* we find the following four actions: ACCEPT, LOG, REJECT, and DROP. The action ACCEPT allows the packet to pass, while REJECT and DROP both block the packet. In addition to blocking the packet, REJECT also sends an error message to the requester, thereby acknowledging the receipt of the packet. Finally, the action LOG simply records the request in the syslog files.

We now consider three network structures to protect the above listed resources. The first network structure, shown in Fig. 1, has an outer firewall protecting all the resources. Ob-



(a) A topological representation of network (2)



(b) A graph representation of network (2)

Fig. 6 – Layered Network Structure (2).

serve that the outer firewall Fw1 does not provide sufficient protection to the engineering and finance resources, as it must have a policy to enable access to Web server and Email server, which in turn allows access requests coming from the Internet. As a result, the segmentation of resources in Fig. 1 leaves the resources with stricter access policy vulnerable (for example Engineering1 and Engineering2). Moreover, there is no protection from any internal attacks. Therefore, there is a need for a better network structure, in which resources are segmented

to achieve better protection from external and internal unauthorized access. The main concern is whether we can detect the weakness of this solution through a simple formal process. Furthermore, in this case we have a simple example for illustration, however is it possible to detect the weaknesses of a large network?

As discussed above, while building a network it is important to adhere to segmentation and DD strategies. The second network structure shown in Fig. 6 attempts to implement both

strategies. However, observe that in this network the outer firewall Fw1 and the internal firewall Fw3 would necessarily have the same policy; both of these firewalls allow traffic from the Internet. As a result, traffic from the Internet will not be blocked by Fw1 or Fw3, making the engineering workstations vulnerable. An attacker from outside can easily gain access to the same subnetwork where the engineering workstations reside. Therefore, although the structure seems to implement segmentation and DD strategies, it does not provide the best protection/segmentation of resources. Simply adding extra layers of firewalls does not always guarantee extra protection without proper policy and proper placement of resources and firewalls. More importantly, we are concerned with the ability to reach these conclusions for large networks, something not readily possible via only general guidelines.

Finally, the third network structure shown in Fig. 7 shows the ideal network architecture, which correctly follows the guidelines and best practices for network design (Peteanu, 2001). Following such guidelines might be easy and achievable for a small size network such as this example. However, for a large network with a huge number of resources, it is almost impossible using only human judgment, motivating a formal approach. We will show later that we can obtain it through calculations.

3. Mathematical background

To formalize network segmentation we use the theory of Product Family Algebra (PFA) which treats policies as a family of related products (in this case sets of policies), and the theory of *Guarded Commands* which deals with rules of policies and their conditions and actions. We then use it and the DD strategy (Khedri et al., 2017) to build a robust secure network. Below we briefly describe these formalisms and their usage in our context.

3.1. Guarded commands

We use a variant of Dijkstra's guarded command introduced in Höfner et al. (2011b) and used in Khedri et al. (2017); Möller and Struth, 2006 to model policies. A command is a transition relation from starting states to their successor states, and a set of states that do not lead to failure, and formally defined as follows: for a set Σ of states, a *command* over Σ is a pair (R, P) , where $R \subseteq \Sigma \times \Sigma$ is a transition relation, and P is a subset of Σ that is intended to characterize those states from which the command cannot lead to abortion/failure. The command *abort* is a command that offers no transition and does not guarantee the absence of abortion/failure for any state, and is defined as: $\text{abort} \stackrel{\text{def}}{=} (\emptyset, \emptyset)$.

For a command (R, P) and a set of states $Q \subseteq \Sigma$, the *guarded command* $Q \rightarrow (R, P)$ (where Q is called the *guard*) is defined as $Q \rightarrow (R, P) \stackrel{\text{def}}{=} (Q \downarrow \text{RRQ}, \overline{Q} \cup P)$, where $Q \downarrow \text{RRQ}$ is the restriction of R to Q defined as $Q \downarrow \text{RRQ} \stackrel{\text{def}}{=} R \cap (Q \times \Sigma)$ and $\overline{Q}$ is the complement of Q w.r.t. Σ . For the set of states from which the guarded command does not lead to abortion, we augment the set P by the complement of Q ; that is, by $\overline{Q}$. The reason is that outside of the set of states Q (i.e., $\overline{Q}$) the command cannot

be executed and therefore there is no possibility of abortion or failure. For a relation $R \subseteq A \times B$, the domain of R that is denoted by $\text{dom}(R)$ is $\{x \mid x \in A \wedge (\exists y \mid y \in B \cdot (x, y) \in R)\}$.

Let $\Sigma = S \times P \times \text{St} \times \text{Dport} \times A$, where S is the set of all possible source IP numbers, P is the set of all possible protocols, St is the set of connection states, Dport is the set of destination ports, and A is the set of actions, respectively. The sets $S, P, \text{St}, \text{Dport}$, and A are called the *state attributes*. We denote by $\mathbb{L}$ the universal relation on this space.

In this setting, the rule on Line 1 of the policy shown in Fig. 2 can be written as follows: $C_1 = [Q \rightarrow (R, P)]$, where $Q \subseteq \Sigma$ is the following guard:

$$\text{gt } \{(s, p, \text{st}, \text{dport}, a) \mid \text{st} \in \{\text{RELATED}, \text{ESTABLISHED}\}\},$$

and $\overline{Q}$ is:

$$\text{gt } \{(s, p, \text{st}, \text{dport}, a) \mid \text{st} \notin \{\text{RELATED}, \text{ESTABLISHED}\}\}.$$

The relation R can be defined in this case as

$$\text{gt } R = \{((s, p, \text{st}, \text{dport}, a), (s', p', \text{st}', \text{dport}', a')) \mid$$

$$\text{gt } a' = \text{ACCEPT}\}.$$

Since the domain of the relation R is Σ with no restrictions, we cannot guarantee the absence of abortion for any state. Therefore, $P = \emptyset$. After restricting R to Q we have:

$$\text{gt } Q \downarrow \text{RRQ} = \{((s, p, \text{st}, \text{dport}, a), (s', p', \text{st}', \text{dport}', a')) \mid$$

$$\text{gt } (\text{st} \in \{\text{RELATED}, \text{ESTABLISHED}\}) \wedge a' = \text{ACCEPT}\}.$$

$$\text{and } C_1 = [Q \rightarrow (R, P)] = [Q \downarrow \text{RRQ}, \overline{Q}].$$

We treat the other rules of the policy analogously.

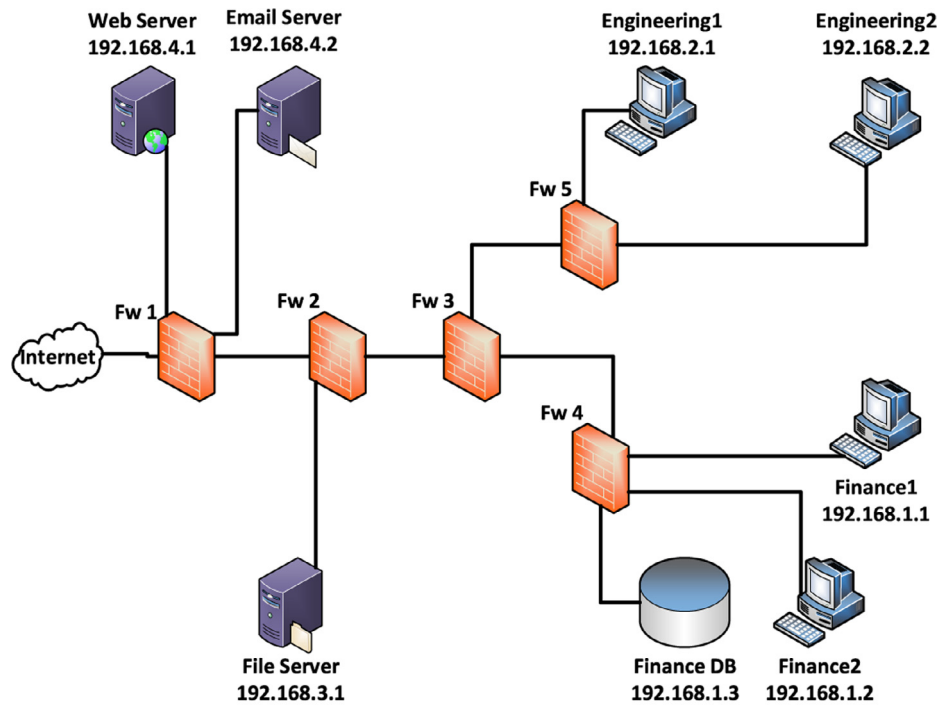
The policy in Fig. 2 can also be represented as a set of rules or a single rule obtained by combining the relations of the rules as follows:

Relations can be represented in many ways. However, we are particularly interested in representing relations as tabular expressions (Janicki and Khedri, 2001; Jin and Parnas, 2010; Parnas, 1992), as they simplify the computation of policies (see Appendix D).

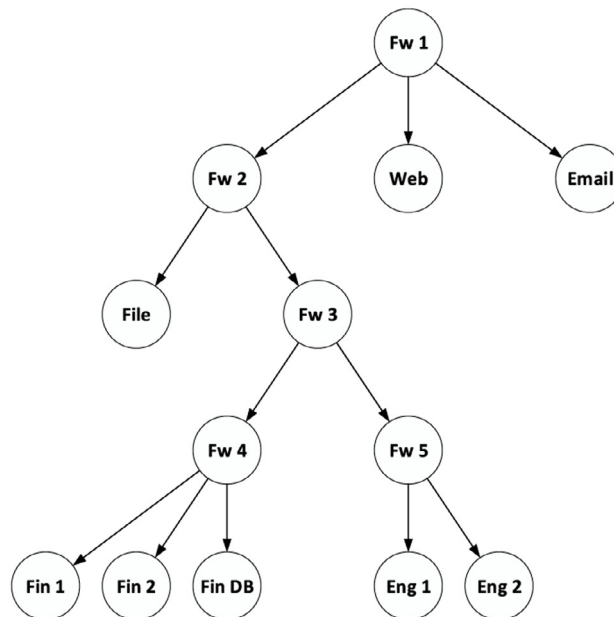
We say that command (R, P) refines command (S, Q) and we write $(R, P) \sqsubseteq (S, Q) \stackrel{\text{def}}{=} Q \subseteq P \wedge Q \downarrow \text{RRQ} \subseteq S$.

If we take the command $C_1 = [Q \downarrow \text{RRQ}, \overline{Q}]$ presented above, we can easily verify that it is refined by $C_2 = [P, \overline{Q}]$, where

The command C_2 is more restrictive than C_1 as it requires a specific port while C_1 does not. In term of policies, a policy refines another if it is more restrictive in its rules or it is more deterministic.



(a) A topological representation of network (3)



(b) A graph representation of network (3)

Fig. 7 – Robust Network Structure (3).

$$\text{gt } R_{web} = \{((s, p, st, dport, a), (s', p', st', dport', a')) \mid$$

$$\text{gt } (st \in \{\text{RELATED}, \text{ESTABLISHED}\}) \wedge a' = \text{ACCEPT}$$

$$\text{gt } \vee (st \notin \{\text{RELATED}, \text{ESTABLISHED}, \text{NEW}\} \wedge a' = \text{DROP})$$

$$\text{gt } \vee (p = \text{TCP} \wedge st = \text{NEW} \wedge dport \in \{80, 25\} \wedge a' = \text{ACCEPT})$$

$$\text{gt } \vee (s \in \{192.168.1.0/24, 192.168.2.0/24, 192.168.3.0/24,$$

$$192.168.4.0/24\}$$

$$\text{gt } \wedge a' = \text{ACCEPT})$$

$$\text{gt } \vee (s \notin \{192.168.1.0/24, 192.168.2.0/24, 192.168.3.0/24,$$

$$192.168.4.0/24\}$$

$$\text{gt } \wedge p \neq \text{TCP} \wedge st = \text{NEW} \wedge a' = \text{DROP})$$

$$\text{gt } \vee (s \notin \{192.168.1.0/24, 192.168.2.0/24, 192.168.3.0/24,$$

$$192.168.4.0/24\}$$

$$\text{gt } \wedge p = \text{TCP} \wedge st = \text{NEW} \wedge dport \notin \{80, 25\} \wedge a' = \text{DROP})\}$$

$$\text{gt } P = \{((s, p, st, dport, a), (s', p', st', dport', a')) \mid$$

$$\text{gt } (st \in \{\text{RELATED}, \text{ESTABLISHED}\})$$

$$\text{gt } \wedge dport' = 25 \wedge a' = \text{ACCEPT})\}.$$

Based on [Desharnais et al. \(1995\)](#), a relation Q refines a relation R written as $Q \sqsubseteq R$ iff $(Q \cap (\text{dom}(R) \times \Sigma) \subseteq R) \wedge (\text{dom}(R) \subseteq \text{dom}(Q))$. Therefore, C_1 given above is refined by C_2 . This is clear from the example given for the demonic join. The demonic join $R_1 \sqcup R_2$ is refined by both R_1 and R_2 .

The *greatest lower bound of commands* (R, P) and (S, Q) w.r.t. $\sqsubseteq$ is the *demonic meet* which is defined as

$$(R, P) \sqcap (S, Q) = ((R \cap S) \cup (Q \downarrow R\bar{S}\bar{P}) \cup (Q \downarrow R\bar{R}\bar{Q}), P \cup Q)$$

The *demonic meet* for the commands $(R, \text{dom}(R))$ and $(S, \text{dom}(S))$ is defined iff $\text{dom}(R \cap S) = \text{dom}(R) \cap \text{dom}(S)$. In other terms, the demonic meet of the commands $(R, \text{dom}(R))$ and $(S, \text{dom}(S))$ is defined iff R and S agree on the action to be carried on their common domain. This property is called *integrability* in [Höfner et al. \(2006\)](#). The *least upper bound of commands* (R, P) and (S, Q) w.r.t. $\sqsubseteq$ is the *demonic join* which is defined in [Desharnais et al. \(1995\)](#) as $(R, P) \sqcup (S, Q) =$

$$((R \cup S) \cap (\text{dom}(R) \times \Sigma) \cap (\text{dom}(S) \times \Sigma), P \cap Q)$$

The demonic join coincides with what we call later the Greatest Common Divisor (GCD). Let R_1 and R_2 be the relations representing policies P_1 and P_2 respectively:

$$\text{gt } R_1 = \{((s, p, st, dport, a), (s', p', st', dport', a')) \mid$$

$$\text{gt } (s = 192.168.1.0/24 \wedge a' = \text{ACCEPT})$$

$$\text{gt } \vee (s = 192.168.2.0/24 \wedge a' = \text{REJECT})$$

$$\text{gt } \vee (s = 192.168.3.0/24 \wedge a' = \text{ACCEPT})$$

$$\text{gt } R_2 = \{((s, p, st, dport, a), (s', p', st', dport', a')) \mid$$

$$\text{gt } (s = 192.168.1.0/24 \wedge a' = \text{ACCEPT})$$

$$\text{gt } \vee (s = 192.168.2.0/24 \wedge a' = \text{ACCEPT})$$

$$\text{gt } \vee (s = 192.168.4.0/24 \wedge a' = \text{ACCEPT})$$

Then, the demonic join of R_1 and R_2 , $R_1 \sqcup R_2 =$

$$\text{gt } \{((s, p, st, dport, a), (s', p', st', dport', a')) \mid$$

$$\text{gt } (s = 192.168.1.0/24 \wedge a' = \text{ACCEPT})$$

$$\text{gt } \vee (s = 192.168.2.0/24 \wedge$$

$$\text{gt } (a' = \text{ACCEPT} \vee a' = \text{REJECT}))$$

For further information on guarded commands, we refer the reader to [Höfner et al. \(2011b\)](#); [Möller and Struth, 2006](#).

3.2. Product family algebra

One way to handle large networks is by abstracting subnetworks into one node that has a family of policies and not one policy as we would do for a single resource. Consider a room including a large number of computers that have very similar access control policies. We can abstract this room into one node in the global network that has a family of policies. Moreover, we can have our global network distributed in many buildings. In designing the subnetwork for a building, we consider the other resources located at other buildings as nodes each with a family of policies. Furthermore, in real world networks, a security policy governing an access control point consists of rules, possibly coming from different stakeholders (for example management, security officers, or users). Clearly, the policies derived from the perspectives of the stakeholders, share common rules and differ on others. Therefore, we can think of these policies as families of policies. Hence, we need to adopt a family approach to manage similar cases illustrated

above. For this purpose, we adopt PFA as the formalism that links the access control policies to the network.

PFA (Höfner et al., 2006; 2008; 2011a) is a commutative idempotent semiring $(S, +, \cdot, 0, 1)$, where S is a set of product families. The binary operators $+$ and $\cdot$ are interpreted as the alternative choice between two product families and the mandatory composition of two product families, respectively. The constant 0 corresponds to an empty product family, and the constant 1 corresponds to a product family consisting of only a pseudo-product with no features; that is, no access control rules. A product is a family that is indivisible with regards to $+$ operator. A feature is a product that is indivisible with regards to $\cdot$ operator. For the mathematical properties of products and features, we refer the reader to Höfner et al. (2008).

We have the divisibility relation among families $(a \mid b) \iff (\exists c \mid \cdot \sim b = a \cdot c)$ ², which allows us to find divisors of families. And therefore, find the GCD, which is the common divisor that is divided by all other common divisors. Hence, the following property holds: $\gcd(a, b) = d$ such that the following condition is satisfied, $[(d \mid a) \wedge (d \mid b) \wedge ((\forall c \mid \cdot \sim (c \mid a) \wedge (c \mid b)) \implies (c \mid d))]$. Finding the commonalities of two families is formalized by finding the GCD. For two families a and b we say they are coprime family if $\gcd(a, b) = 1$. For any family a , $\gcd(1, a) = 1$, and $\gcd(a, 0) = a$, as we have $a \cdot 0 = a$ and therefore $a \mid 0$.

In the context of access control policies, an atomic rule is a feature (and hence indivisible) and is modelled as a guarded command. A policy is a set of rules or a single rule obtained by combining various rules.

3.2.1. Firewall policies as product families

Let $\mathbb{P}$ be a set of policies. Then, an element of $\mathcal{P}(\mathbb{P})$ is a family of policies. In Khedri et al. (2017) the commutative idempotent structure $\mathcal{F}$ is presented as a model for PFA as follows: $\mathcal{F} = (\mathcal{P}(\mathbb{P}), \oplus, \odot, 0_{\mathcal{F}}, 1_{\mathcal{F}})$ is a product family algebra, where for every $A, B \in \mathcal{P}(\mathbb{P})$, we have

1. $A \oplus B \stackrel{\text{def}}{=} A \cup B$
2. $A \odot B \stackrel{\text{def}}{=} \{a \sqcap_{\mathbb{P}} b \mid a \in A \wedge b \in B\}$
3. $0_{\mathcal{F}} \stackrel{\text{def}}{=} \emptyset$
4. $1_{\mathcal{F}} \stackrel{\text{def}}{=} \{\{\text{abort}\}\}$

where $\oplus$ is the union of the family of policies and represents the choice of families, and $\odot$ is an extended notion of the demonic meet presented previously on the policies of the families and it represents the integration of families. The family $0_{\mathcal{F}}$ is the annihilator element, and $1_{\mathcal{F}}$ is the neutral element for the $\odot$ operator.

The families $0_{\mathcal{F}}$ and $1_{\mathcal{F}}$ are two pseudo families of policies. The family $0_{\mathcal{F}}$ is the empty set of policies. The singleton family $1_{\mathcal{F}}$ includes only one policy that is modelled with the command abort. This policy does not transform the state space and does not have any actions to carry. Moreover, its integration

with another policy P preserves P (i.e., neutral for policy integration as $P \sqcap_{\mathbb{P}} \text{abort} = P$). For instance, if we have a policy that has no rules, we can model it with the pseudo command abort. The family that includes only one “policy with no rules” can be modelled with $1_{\mathcal{F}}$. Hence, when a resource has no restrictions and open to all users, we indicate that it is protected with the family of access policies $1_{\mathcal{F}}$.

On a product family, $\preceq_{\mathcal{F}}$ denotes the natural order that comes with the semiring structure for $\mathcal{F}$. It is defined as $a \preceq_{\mathcal{F}} b \stackrel{\text{def}}{=} a \oplus b = b$. Then the notion of family refinement of the elements of $\mathcal{F}$ is defined as follows: $a \sqsubseteq_{\mathcal{F}} b \stackrel{\text{def}}{=} (\exists c \mid \cdot \sim a \preceq_{\mathcal{F}} b \odot c)$.

The GCD defined under PFA is equivalent to the demonic join of the families of policies and can be restated as follows: $(\forall A, B \mid A, B \in \mathcal{P}(\mathbb{P}) \cdot \gcd(A, B) \stackrel{\text{def}}{=} \{a \sqcup_{\mathbb{P}} b \mid a \in A \wedge b \in B\})$, where $\sqcup_{\mathbb{P}}$ is the extension of $\sqcup$ to accommodate families of policies, introduced in the definition of demonic join in Section 3.1.

3.3. Defence in depth

A network with an outer access control point r ; that is, a firewall, can be represented as a directed acyclic graph. The leaves represent the resources and the internal vertices represent firewalls. Fig. 8 shows the graph model of a network that has a root r , internal access points v_1 to v_5 , and leaf nodes v_6 to v_{10} . The edges represent traffic links. For example in Fig. 8, the edge (v_2, v_3) indicates the connection from v_2 to v_3 .

Let $G \stackrel{\text{def}}{=} (V, E, r)$ be a rooted connected directed acyclic graph that represents a resource network. The set V denotes the set of vertices or access control points that enforce access policies (i.e., firewalls and resources). The set E is a set of ordered pairs of vertices that represent the link between access control points. The vertex r is the root of the graph and it represents the access point between the network and the external world.

To guarantee the absence of conflicts in the policies of a network G , any edge between two access control points should respect the refinement property discussed in Section 3.1. For example if the edge $(v_2, v_3) \in E$ then the family of policies implemented at v_2 is refined by the policy implemented at v_3 . Therefore, the family of policies at a node is a refinement of its ancestors.

Definition 1 (Defence in Depth Law (DDL) (Khedri et al., 2017)). Let $G \stackrel{\text{def}}{=} (V, E, r)$ be a network of access control points. We denote by $p(v)$ the family of policies enforced at vertex v in G . The network G employs a DD strategy if

Let $G \stackrel{\text{def}}{=} (V, E, r)$ be a network of access control points, and let T be a spanning tree of G rooted at r . Let L be the set of leaf nodes, where each leaf $l \in L$ is assigned a policy $p(l)$. We use the gcd operator scheme presented in Khedri et al. (2017) to define $p(v)$ as follows: $p(v) = (\gcd v_i \mid (v, v_i) \in E \cdot p(v_i))$ ³

² In this paper, we adopt the notation used by Gries and Schneider in Gries and Schneider (1993) for quantifiers. The general form of the notation is $\star(x \mid R : P)$ where $\star$ is the quantifier, x is the dummy or quantified variable, R is predicate representing the range, and P is an expression representing the body of the quantification. An example of the notation is

³ If the range of the quantification is empty then the quantification returns the neutral value $\mathbb{L}$ for gcd; that is, $\gcd(x, \mathbb{L}) = x$, which is the universal relation on the space of the policies, as mentioned in Section 3.1.

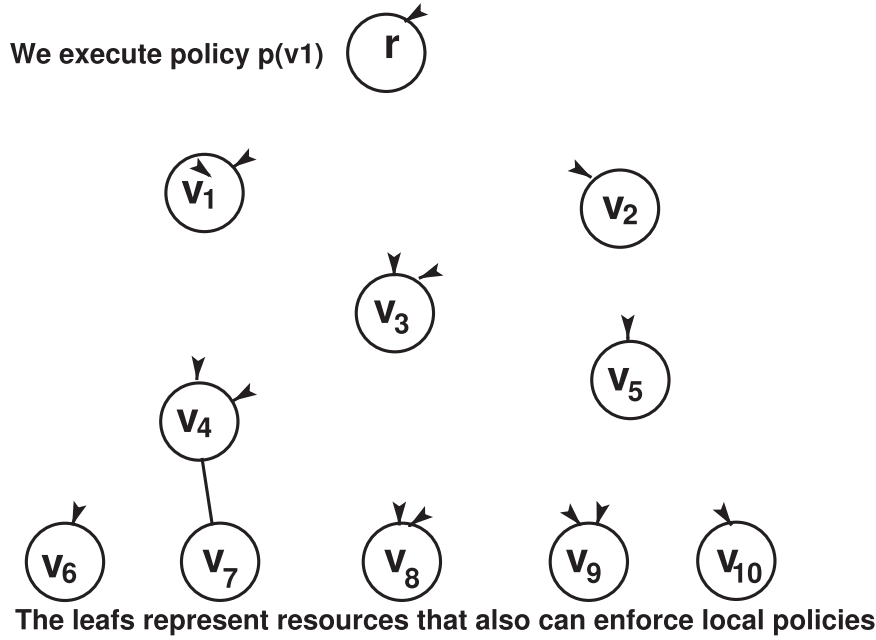


Fig. 8 – A Network as a rooted connected directed acyclic graph (figure borrowed from Khedri et al. (2017)).

Obviously, a network G could have multiple spanning trees $T_1 \dots T_n$, where for each tree T_i , $1 \leq i \leq n$, the family of policies at an internal node is computed using the gcd operator scheme. Also note that, an internal node could be a node in multiple spanning trees, say $T_i \dots T_k$. In this case, the family of policies implemented at such a node v is $p(v) = (\gcd \{ p_i(v) \mid 1 \leq i \leq k \})$, where $p_i(v)$ is the family of policies for the node v in the spanning tree T_i .

4. Strict defence in depth as a base for our robust network architecture

In this section, we discuss the Strict Defence in Depth strategy SDD which is a more strict form of DD strategy (Definition 1) achieved by strengthening the refinement condition. We use the SDD strategy to place nodes in our robust network graph.

Definition 2 (SDD (Khedri et al., 2017)). Let $G \stackrel{\text{def}}{=} (V, E, r)$ be a network of access control points. The network G employs an SDD strategy if

$$p(r) \neq 0_{\mathcal{F}} \wedge (\forall a, b \mid (a, b) \in E \cdot p(b) \sqsubset_{\mathcal{F}} p(a)),$$

$$\text{where } p(b) \sqsubset_{\mathcal{F}} p(a) \iff (p(b) \sqsubseteq_{\mathcal{F}} p(a) \wedge p(a) \neq p(b)).$$

The SDD is a stronger form of DD. We obtain SDD from DD by replacing $\sqsubseteq_{\mathcal{F}}$ by its stricter form $\sqsubset_{\mathcal{F}}$. By using SDD to build the network graph we exclude the case where the families of policies on successive nodes are equal. This allows us to ensure that firewalls with the same policies do not exist in any path from the root to a leaf node.

Let $G \stackrel{\text{def}}{=} (V, E, r)$ be a network of access control points and $S \stackrel{\text{def}}{=} \{v \mid v \in V \wedge (r, v) \in E\}$ be the set of vertices connected to the root r . Let T be a spanning tree of G rooted at r . Every leaf node $l \in L$ is assigned a family of policies $p(l)$. If we want to generate the families of policies of the internal vertices such that G implements SDD, the below lemmas present cases when SDD is not achievable in the network. The proofs of these lemmas are in A.1.

Lemma 1. Let T be a spanning tree of G having a height greater than 2. If T has a leaf $l \notin S$, such that $p(l) = 1_{\mathcal{F}}$, then it is impossible to have an SDD implementation on G .

Formally,

$$(\exists l \mid l \in L \cdot p(l) = 1_{\mathcal{F}})$$

$\implies$

$$\neg(p(r) \neq 0_{\mathcal{F}} \wedge (\forall a, b \mid (a, b) \in E \cdot p(b) \sqsubset_{\mathcal{F}} p(a)))$$

Lemma 1 states that if a leaf node exists such that it is not directly attached to the root and has a family of policies equal to $1_{\mathcal{F}}$, then the family of policies executed at its ancestors will be $1_{\mathcal{F}}$. For example in Fig. 9, if $p(v_6) = 1_{\mathcal{F}}$ then the policies at nodes v_2 and r will also be $1_{\mathcal{F}}$; that is $p(v_2) = 1_{\mathcal{F}}$ and $p(r) = 1_{\mathcal{F}}$. This case does not meet the requirements of the SDD strategy which states that each child node should strictly refine its parent.

Having a leaf node with a family of policies equal to $1_{\mathcal{F}}$ deep in the network will result in no security control in the chain of firewalls from the root to that node. Thereby allowing unauthorized users access deep into the network without any line of defence. Consequently, this will hinder the overall security of the network. To prevent this case we need to iso-

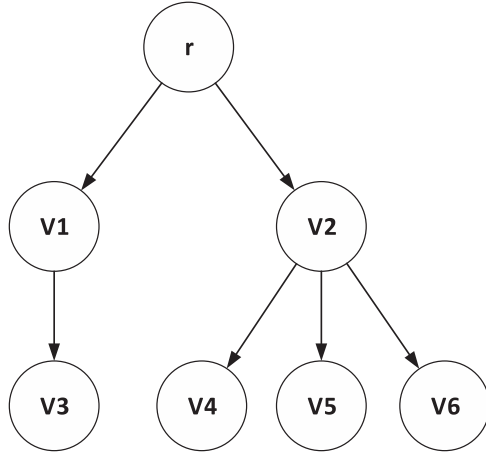


Fig. 9 – A network graph.

late the nodes having $1_{\mathcal{F}}$ and move them closer to the outer edge. A resource with $1_{\mathcal{F}}$ is open to be accessed by everyone. Therefore by segregating and moving the nodes with $1_{\mathcal{F}}$ close to the root we can protect the resources with stricter policies efficiently.

A resource protected by $1_{\mathcal{F}}$ is interpreted as being protected by a policy with no rules (an empty policy). Therefore, it is open to any user. Hence, it should be attached to the root. Any resources with stricter policy than $1_{\mathcal{F}}$ should be at a level higher than one (assuming the level of the root is zero). Resources in what is commonly referred to as DMZ are resources that have some level of protection stricter than $1_{\mathcal{F}}$. However, they are primarily public facing resources. Hence they should be connected to the root or to a child of the root.

Lemma 2. Let T be a spanning tree of G having a height greater than 2. If T has two leaves l_1, l_2 belonging to the same subtree having coprime families of policies, then it is impossible to have an SDD implementation on G .

Formally,

$$\begin{aligned}
 &(\exists l_1, l_2, s, n, m \mid s \in S \wedge n \geq 1 \wedge m \geq 1 \wedge (s, l_1) \in E^n \\
 &\wedge (s, l_2) \in E^m \cdot \gcd(l_1, l_2) = 1_{\mathcal{F}}) \\
 &\implies \\
 &\neg(p(r) \neq 0_{\mathcal{F}} \wedge (\forall a, b \mid (a, b) \in E \cdot p(b) \sqsubset_{\mathcal{F}} p(a)))
 \end{aligned}$$

$(a, b) \in E^k$ indicates a path with k edges from a to b . **Lemma 2** states that if a subtree of T has two nodes such that their families of policies have nothing in common, then the family of policies of their common ancestors, including the root of the subtree and the root of the tree, is equal to $1_{\mathcal{F}}$. In this case, the SDD strategy is not satisfied. For example, in Fig. 9, if $p(v_4)$ and $p(v_6)$ are coprime families of policies such that $\gcd(p(v_4), p(v_6)) = 1_{\mathcal{F}}$, then $p(v_2) = 1_{\mathcal{F}}$ and $p(r) = 1_{\mathcal{F}}$.

Having nodes with coprime families of policies under a subtree will result in no security control in the chain of firewalls from their common ancestor to the root. This in turn affects the overall security of the network. Therefore a well

designed network adhering to the SDD, groups resources that share common policies (e.g., resources belonging to the same department) in a common segment, and places resources having nothing in common into different segments/subnetworks. This approach allows for greater security and protection than grouping nodes randomly.

Lemma 3. If we have a family of policies at a node that is equal to the GCD of the family of policies of its siblings, then it is impossible to have an SDD implementation on G .

Formally,

$$\begin{aligned}
 &(\exists v \mid (u, v) \in E \cdot p(v) = (\gcd v_i \mid (u, v_i) \in E \cdot p(v_i))) \\
 &\implies \\
 &\neg(p(r) \neq 0_{\mathcal{F}} \wedge (\forall a, b \mid (a, b) \in E \cdot p(b) \sqsubset_{\mathcal{F}} p(a)))
 \end{aligned}$$

Lemma 3 states that if we have a node such that its family of policies is equal to the commonality of its siblings, then the family of policies of its parent is the same as that of the node. Clearly in this case the SDD strategy is not satisfied. For example in Fig. 9, if $p(v_4) = \gcd(p(v_5), p(v_6))$, then the family of policies of their parent v_2 ; that is, $p(v_2) = p(4)$.

Based on the above lemmas we propose an approach to network segmentation that implements SDD in the following section.

5. Network segmentation and robust network architecture

In this section, we formally define the notions of network segmentation and robust network architecture. We introduce a weight function to quantify the different security requirements of an access control policy. We then use this function to formally define network segmentation.

Recall that an atomic rule is modelled as a guarded command, which is a transition relation from the starting state to end state(s). Therefore, we compute the weight of an atomic rule based on the weights of the end state(s). One way of doing this is by assigning weights to the different values of chosen state attributes and using them to compute the weight of an end state, such that the values having higher security requirements are assigned higher weights. Furthermore, since a chain is as secure as its weakest link, an atomic rule with a relation that maps a starting state to multiple end states has a weight equal to the minimum weight of its end states.

Formally, let SA_i , where $1 \leq i \leq m$, be the different state attributes, and let $V_{SA_i} = \{a_{i1}, a_{i2}, \dots, a_{in}\}$ be the set of all possible values assigned to SA_i . Then $w_{V_{SA_i}} : V_{SA_i} \rightarrow \mathbb{Z}$ is the weight function that assigns an integer value to each element in V_{SA_i} , such that for any two elements $a_{ik}, a_{il} \in V_{SA_i}$ if the security requirement of a_{ik} is less than that of a_{il} , then $w_{V_{SA_i}}(a_{ik}) < w_{V_{SA_i}}(a_{il})$. Let $\mathbf{R}$ be the set of all atomic rules for all the resources in the organization. Then $w_{\mathbf{R}} : \mathbf{R} \rightarrow \mathbb{N} \cup \{-1\}$, is the weight function which assigns to an atomic rule $r \in \mathbf{R}$ its corresponding weight. The weight of $1_{\mathcal{F}}$ is taken to be -1 , as it is a rule that does not bring any security constraints. Furthermore, the

weight of a rule (r) with its domain mapped to multiple end states (say p), is the minimum of the weights assigned to its end states. For each end state s_i , where $1 \leq i \leq p$, we take $v_{s_i} = \text{eval}(w_{V_{SA_1}}, w_{V_{SA_2}}, \dots, w_{V_{SA_p}})$, where $w_{V_{SA_i}}$ is the weight of the assigned value to the state attribute SA_i . For a state s_i , the *eval* function takes the weight of the attributes of s_i , and assigns to it and overall security weight v_{s_i} . Then to compute the weight of such an atomic rule, we take the minimum of all the values v_{s_i} . Therefore, for an atomic rule r that has p end states, we have $w_R(r) = \min(v_{s_1}, \dots, v_{s_p})$.

For the illustrative example, we compute the weights of our atomic rules based on the state attribute ACTION (AC). The set of values of this attribute as given by the set $AC = \{\text{ACCEPT}, \text{REJECT}, \text{DROP}\}$. As discussed in Section 1, the three actions in AC contribute differently to the confidentiality of the resources of the network, and so DROP has a higher weight than REJECT, which in turn has a higher weight than ACCEPT. Based on this we assign the following weights to each element in AC: $w_{AC}(\text{ACCEPT}) = 0$, $w_{AC}(\text{REJECT}) = 1$, and $w_{AC}(\text{DROP}) = 2$. Then for example, a rule that maps an initial state to two end states (s_1 and s_2) where the action attribute in s_1 is assigned the value ACCEPT and the action attribute in s_2 is assigned the value REJECT, then the rule has a weight equal to the minimum weight of the two; that is, 0.

The weight of a policy or a combined rule r that is formed by the set A_r of atomic rules is the sum of the weights of the atomic rules in A_r . Let P be the set of all policies composed of the atomic rules in R . Then $w_P : P \rightarrow \mathbb{N} \cup \{-1\}$, is the weight function that assigns an integer value to each element in P based on w_R . We assign -1 as the weight of the 1_F policy, and $+\infty$ as the weight of the 0_F policy.

In a typical firewall policy, rules are executed sequentially. Such rules usually are not atomic. For example, Rule 9 in the policy of Fig. 2 can be divided by all other rules in the policy. Therefore, a rule might be followed by another that includes it. If we apply the weight function on these rules, we can have double counting of weights. Therefore, to avoid any problem we transform policies into a set of rules that are (relative) atomic (or, relative prime). A *relative atomic/prime rule* in this context is a rule that is indivisible by any other given rule (i.e., it is only divisible by 0 and itself). The weight function takes such a rule and assigns a weight to it. For example, consider the policy shown in Fig. 4 which is transformed to a relative atomic policy with 270 rules: two rules with ACCEPT actions, three rules with REJECT actions, and 265 rules with DROP actions. Therefore, the weight assigned to the policy is $2 * 0 + 3 * 1 + 265 * 2 = 533$. Fig. 16 in Appendix E shows a snippet of the relative atomic policy. These generated relative atomic rules are only to calculate the weight and identify the commonalities among policies. Once the commonalities are calculated we discard them.

As can be seen we measure the level of security requirements of a resource by the weight of the policy governing it, such that its security requirements are directly proportional to the weight of its policies. For example, consider two resources v_1 and v_2 , where the weights of their policies are $w_P(p(v_1))$ and $w_P(p(v_2))$, respectively. If $w_P(p(v_2)) < w_P(p(v_1))$, it means that v_1 has higher level of security requirement than v_2 .

Let w_R be the partial order on the atomic rules based on their weights and w_P be the total order on the policies based

also on their weights. Then there exists an order preserving map $f : w_R \rightarrow w_P$ satisfying the following condition:

$$\begin{aligned} &(\forall i \mid 1 \leq i \leq n \wedge x_i, y_i \text{ are atomic rules} \cdot x_i <_{w_R} y_i) \\ &\Leftrightarrow f(P(x_1, x_2, \dots, x_n)) <_{w_P} f(P(y_1, y_2, \dots, y_n)), \end{aligned}$$

$P(x_1, x_2, \dots, x_n)$, and $P(y_1, y_2, \dots, y_n)$ are the policies consisting of atomic rules $x_1, x_2, \dots, x_n$ and $y_1, y_2, \dots, y_n$ respectively. Since w_R is a partial order, some values of the atomic rules might not have a weight assigned. In this case, the mapping f assigns zero, which is the neutral value for addition.

We now define segmentation formally based on the weights of the commonalities among the policies of the resources in a segment.

Definition 3 (Segment). Let R be a set of resources. A set $S \subseteq R$ is said to be a segment of R iff

$$gt(\forall r, r' \mid r \in S \wedge r' \in (R - S) \cdot w_P(\text{gcd}(p(r), p(r')))) \leq$$

$$gt(w_P(\text{gcd}(r \mid r \in S \cdot p(r)))).$$

Our definition of a segment forces us to have at least two elements in a segment, as the GCD is a 2-arity operator.

Definition 4 (Segmentation). Let R be a set of resources, and let F be a set of subsets of R such that $(\cup A \mid A \in F \cdot A) = R$. Then F is a segmentation of R iff $(\forall A \mid A \in F \cdot A)$ is a segment of R .

In our definition of segmentation, a resource is placed within a group of resources if and only if the commonalities it has with the members of the group is more strict than with any outside resource; that is, the commonalities have equal or more weight than the weight of the commonalities it has with any other resource not in the segment. As a result, this segmentation provides maximum access-protection to its resources.

Lemma 4.

- (a) Let $S \subseteq R$ be a set of resources formed by only mutually co-prime policies that are $\neq 1_F$. S forms a segment only if $(\forall r \mid r \in (R - S) \cdot p(r) = 1_F)$.
- (b) Let $S \subseteq R$, and there exists resources $r, r' \in S$ such that $p(r) = 1_F$ and $p(r') \sqsubset 1_F$, and

$$(\forall r, r' \mid r \in S \wedge r' \in (R - S) \cdot \text{gcd}(p(r), p(r')) = 1_F).$$

Then S is a segment.

By requiring DD, resources with 1_F policy are directly attached to the root having 1_F policy, thus forming elements of a DMZ. If a segment satisfying the condition in Lemma 4(a) exists then the root of this segment will have 1_F policy. Consequently, the root of the global network will have 1_F policy. To

achieve SDD all the resources and internal nodes attached to the root of the segment S will be attached to the global root. By Lemma 4(b) S is not a segment, if $S \not\subseteq R$, or there do not exist resources $r, r' \in S$, such that $p(r) = 1_F$ and $p(r') \sqsubset 1_F$, or there exists resources $r \in S$ and $r' \in (R-S)$ such that $\gcd(p(r), p(r')) \neq 1_F$. This means that if S is a segment that has some resources with 1_F policy and others resources with policies different than 1_F , then the commonality of each resource in S with a resource out of S has to be 1_F . Otherwise, S cannot be a segment. The proof of Lemma 4 is in A.1.

Superfluous Firewall Chaining exists in a network when a firewall has only a single firewall attached to it. For example, when firewalls are connected forming a chain, and resources are attached only at the bottom of the chain, thus in this case firewalls only exist to protect other firewalls. Therefore, superfluous firewall chaining is a waste of network resources, as it is possible to replace all the firewalls forming a chain with the firewall at the bottom of the chain with resources attached, as it has the most restrictive policies among all the other firewalls in the chain.

We now define a robust network, which provides maximum access-protection to its resources using minimum firewalls. It achieves this by implementing the SDD strategy and segmentation defined in Definition 4.

A network graph G is said to be (access control) *robust* if the following criteria hold:

1. G satisfies SDD strategy in every path from the root to the parent of a resource,
2. G has a segmentation as defined in Definition 4, and
3. G has no superfluous firewall chaining.

The first criterion for a robust network ensures that any two internal nodes strictly refine each other. However, it allows a leaf node and its parent to have the same policy. This can be seen when resources have 1_F policy, and as a result the firewall protecting them; that is, the root will also have 1_F policy. The second criterion ensures that resources are segmented in a way such that maximum access-protection is provided to them. These segments are then placed in the network at varying depths depending on their levels of security requirements. Segments with high levels of security requirements are placed deep down in the network and protected by layers of firewalls each adding an extra level of security. And the segments with low levels of security are placed closer to the root; that is, the outer firewall. Therefore, traffic going from the Internet to an internal segment is faced by layers of firewalls and traffic from segment to segment is managed by internal firewalls. Consequently, segments consisting of resources with high levels of security requirements are protected from internal and external threats. Furthermore, if an unauthorized agent gains access to an internal segment, it will not be able to easily gain access to another segment. Finally, the third criterion ensures that we have the most effective strict defence in depth and segmentation at a minimum cost.

6. Algorithms to build a robust network

In this section, we present two algorithms to build a robust network. In Section 6.1 we present the Exp-RNS algorithm which uses a brute force approach to build a robust network, resulting in an exponential running time. The purpose of presenting this algorithm is that it is easy to grasp the use of the formalism presented in Section 3 in constructing the network graph. Then, in Section 6.2, we present the RNS algorithm that uses a similar approach presented in the BUILD-NETWORK-GRAPH procedure of the Exp-RNS algorithm. However, to achieve a polynomial running time, it uses a bottom up approach to build the network graph and uses resources and their weights as a guide throughout the network building process.

6.1. Exponential network segmentation algorithm: A Brute force approach

The Exp-RNS algorithm (Algorithm 1) uses R to build a robust network by computing the GCD of resource policies and using the refinement relation on these GCDs. For simplicity, we discuss the algorithm using singleton families of policies, and so we refer to them as policy rather than family of policies.

A resource having 0_F is essentially inaccessible, as it cannot be protected through an access control system. The 0_F which represents a pseudo policy cannot be enforced at an access control point. Therefore, it cannot be part of a robust network architecture. Hence, we assume that the input R does not contain resources with 0_F policies. Furthermore by Lemma 1, having any internal node/leaf with 1_F policy implies that SDD is not achievable in the network graph. By Lemma 4(a) resources with 1_F policy have to be attached to the root with 1_F policy. Therefore in the Exp-RNS algorithm, we remove resources with 1_F policy a priori from the set R , and as a final step add these resources to the root.

The Exp-RNS algorithm consists of five main tasks. The first task is calculating the GCDs or common policies. The second task is building a network graph based on the refinement relation consisting of only firewalls. The third task is adding resources to the network graph, and the fourth task is pruning the network graph to remove superfluous firewall chaining. Finally, the fifth task is to add all the resources with 1_F policy (if they exist) to the graph.

Below we explain the five main tasks of the above algorithm and present detailed algorithms for each task in Appendix B for completeness.

Calculating GCDs: The COMPUTE-GCD function computes the commonalities or the GCD of policies of resources. As seen in Section 5, to segment resources we need to place them within a segment such that maximum security (access-protection) is achieved. To achieve this, we use a brute force approach to compute all possible segments by computing the power set of R , $\mathcal{P}(R)$, (where each set in $\mathcal{P}(R)$ represents a possible segment). Then we compute the commonalities in each segment by computing the GCD of the resource policies in that segment. For details on the COMPUTE-GCD function we refer the reader to Appendix B.

Building a Network Graph: The BUILD-NETWORK-GRAPH function builds a network graph G with no resources attached. This graph is a temporary network graph consisting of only firewalls and might contain redundant firewalls. However, the redundant firewalls are removed at a later stage (by the OPTIMIZE-NETWORK-GRAPH procedure). The function uses the GCD set = F , and the refinement relation to produce the network graph. The vertices of the network graph are partitioned into *gcd nodes* representing a *gcd* object (representing firewalls), and *resource nodes* representing resources in the network. Each node in the graph has all the attributes defined for a *gcd* object, and in addition to these it has two more attributes: *is_resource* and π . The attribute *is_resource* identifies whether the node is a resource node or a *gcd* node. It is set to *true* if the node represents a resource, otherwise it is set to *false*. The attribute π stores the set of parent(s) of the node.

The root r is the *gcd* object having the least weight and its set consists of all resources (except the resources with 1_F policy). It is first added to the network graph G . After which nodes in F having the same weight as the root r are removed. Then, it computes $w_{\max}$ weight - the maximum weight over the weights of all nodes in F , and computes the set T consisting of all nodes with $w_{\max}$ weight and removes them from F . Then each node in the set T is added to G using the ADD-NODESET-TO-G procedure. The last two steps are repeated till $F = \emptyset$; that is, the last two steps are repeatedly executed to (possibly) add all the nodes in F to G . For details on the BUILD-NETWORK-GRAPH function and the ADD-NODESET-TO-G procedure, we refer the reader to [Appendix B](#).

Adding resources to the Network Graph: The procedure ADD-RESOURCES-TO-NET adds resource nodes to the network Graph G . While adding the resource nodes, we first order the *gcd* objects in GCD in descending order by weight ($GCD_{\max}$). Then we choose the *gcd* node having the maximum weight and attach all the resources in its set attribute, and in R' (initially $R' = R$), to it. After which we remove the attached resources from the set of resources R' . We do this for each *gcd* node in $GCD_{\max}$. In the end, all resources are added to the network graph G . Observe that by adding resources to *gcd* nodes starting with maximum weight to minimum weight, we ensure that resources with higher security requirements are protected with maximum layers of protection. The ADD-RESOURCES-TO-NET procedure is presented in [Appendix B](#).

Optimize GCD graph: The network graph obtained after executing the procedure Add-Resources-To-Net could suffer from superfluous firewall chaining. To eliminate this issue, we use the OPTIMIZE-NETWORK-GRAPH procedure. This procedure performs a post-order traversal of G using two stacks S , T , so that all children of a node are evaluated before it gets evaluated. Since our graph G is not necessarily binary, we do not always evaluate the children of a node from left to right (any order is acceptable). The only criteria we follow is that all children of a node are evaluated before it. S is used to keep track and evaluate all children of a node before itself, and T is used to store the post-order traversal of G . When S is empty (and T is full), we pop nodes from T one at a time and begin evaluating it. When a node u is being evaluated, if u is a *gcd* node and it has no children or if u has only one child that is a *gcd* node, then u and its corresponding edges are deleted from G . If u has only one child that is a *gcd* node, then the child is attached to u 's

parent. The OPTIMIZE-NETWORK-GRAPH procedure is presented in [Appendix B](#).

Adding resources with policy 1_F to G If $S \neq \emptyset$; that is, the set of resources having 1_F policy is not empty, we add it to the network graph using the procedure ADD-1F-RESOURCES outlined as follows:

1. If the policy of the root r is equal to 1_F , then we directly attach all the resources in the set S to r .
2. However, if the policy of the root r is not equal to 1_F , then we create a new node r' , set its attributes appropriately (as shown in Procedure ADD-1F-RESOURCES), and add it to G . After which we attach r and all the resources in the set S to it. The node r' is the new root for the network graph G .

The ADD-1F-RESOURCES procedure is presented in [Appendix B](#).

Exp-RNS Algorithm: Robustness and complexity

The running time of the Exp-RNS algorithm is exponential in the size of the set of resources R . This is due to the brute force approach used to compute the *gcd* nodes, obtained by computing the power set $\mathcal{P}(R)$, and using it to build the network graph.

Theorem 1. *The Exp-RNS Algorithm constructs a robust network graph.*

Proof. The proof is straight forward and is by contradiction. We assume that at least one of the three criteria required for a network graph to be robust ([Section 5](#)), is not satisfied.

Suppose Condition 1 is not satisfied. Then there exists at least one path in G from the root to the parent of a resource, such that the SDD strategy is not satisfied. Recall that the GCD set computed by the COMPUTE-GCD procedure has *gcd* objects with distinct policies. In Exp-RNS, *gcd* nodes are added to G only by the BUILD-NETWORK-GRAPH and ADD-1F-RESOURCES procedures. In the BUILD-NETWORK-GRAPH procedure, a *gcd* node s is first added to G , and any child of r that refines s is disconnected from the root and attached as a child to s . In both these cases, the SDD strategy is satisfied. In the ADD-1F-RESOURCES procedure, the only *gcd* node added to G is the root r' having 1_F policy, and is added only if $r.policy \neq 1_F$. Clearly, SDD strategy is satisfied as all policies $\neq 1_F$ strictly refine 1_F . Therefore, every path from the root to a resources' parent satisfies the SDD strategy, which is a contradiction.

Suppose Condition 2 is not satisfied. Then there exists a segment S , such that a resource $r \in S$ and a resource $r' \notin S$ have commonalities with more weight than the weight of the commonalities between elements of the segment S . If this holds then there will be a *gcd* node in G with the policy $gcd(r, r')$ and its corresponding weight. However, since we add resources to *gcd* nodes starting with *gcd* nodes having maximum weight to the *gcd* nodes having minimum weight, the resources r and r' would be added to the node having the policy $gcd(r, r')$, which is a contradiction.

Suppose Condition 3 is not satisfied; that is, superfluous firewall chaining exists in G . Then there exists at least one firewall with a single node that is a firewall attached. Recall that in the procedure OPTIMIZE-NETWORK-GRAPH, we count the number of children attached to each firewall. If the node has only one child that is a firewall we remove this node and at-

tach its child to its parent(s). Therefore, by the end of the Exp-RNS algorithm G has no superfluous firewalls, which is a contradiction. $\square$

6.2. Polynomial algorithm to build a robust network segmentation

In this section we present the RNS algorithm (Algorithm 2) to

Algorithm 2 Robust Network and Segmentation (RNS) Algorithm.

```

1: procedure SEGMENTATION( $R$ )  $\triangleright R = \{r_1, r_2, \dots, r_n\}$ 
2:    $G \leftarrow \text{NULL}$   $\triangleright G = (V, E, r)$ 
3:    $r \leftarrow \text{CREATE-NODE}(R)$   $\triangleright$  Create root  $r$ 
4:    $\text{ADD-NODE-TO-G}(G, r, \emptyset, \text{false})$   $\triangleright$  Add root  $r$  to  $G$ 
5:    $S_1, S_2, \dots, S_m \subset R$  such that  $\bigcup_{i=1}^m S_i = R$ , and no two subsets
     have resources with same policies
6:    $F = \emptyset$ 
7:   for each  $s \in \{S_1, S_2, \dots, S_m\}$  do
8:      $F = F \cup \text{CREATE-NODE}(s)$ 
9:   end for
10:  while  $F \neq \emptyset$  do
11:     $w_{\max} \leftarrow$  maximum weight of any  $s \in F$ .
12:     $T \leftarrow \emptyset$ 
13:    for each  $s \in F$  do
14:      if  $s.\text{weight} = w_{\max}$  then
15:         $T \leftarrow T \cup s$ ;  $F \leftarrow F - s$ 
16:      end if
17:    end for
18:     $\text{ADD-NODESET-TO-G}(G, F, T, w_{\max})$ 
19:  end while
20: end procedure

```

build a robust network graph G , given a set of resources with its policies R , in polynomial time. Similar to Exp-RNS, for this algorithm also we assume that R does not contain resources with 0_F policies.

The idea of the algorithm is simple. We use an approach similar to the one used in the BUILD-NETWORK-GRAPH procedure presented in Section 6.1, where the gcd nodes are added in non-increasing order of weights (except for the root). However, to achieve a polynomial running time, we use resources and their weights and create temporary nodes to guide us through the network building process.

In the RNS algorithm, the root is added first to G , and nodes are added in batches (set T) to G in decreasing order of their weights. Hence at any given time, when a node s is evaluated to see where in G it can be added, the weight of all the nodes in G is greater or equal $s.\text{weight}$. While adding s to G , clusters of resources are evaluated to see if they can form a segment containing s based on their weights. Note that, the weight of any segment containing s is always less than or equal to $s.\text{weight}$, and so the maximum weight of any segment containing s is $s.\text{weight}$. Therefore, when such a cluster of nodes is identified, we create a gcd node to protect this segment and add it to the root of G and attach the nodes forming the segment to it. Otherwise, we create a temporary node for the cluster and add it to F , so that it can be evaluated later when the set (T) having the weight of the cluster is being evaluated.

Temporary nodes are never added to G . While evaluating a temporary node, if the cluster of nodes with s actually forms a segment at that point, a new gcd node is created and added to G . This is similar to any other gcd node. We now give the outline of the algorithm as follows:

- The algorithm first creates the root r for all resources in R using the CREATE-NODE procedure and adds it to the network graph G .
- Then it creates subsets $S_1, S_2, \dots, S_m \subset R$, such that no two subsets have resources with same policies, and creates a node for each subset S_i , $1 \leq i \leq m$, and adds it to F . Therefore, F contains either a single resource node, or a gcd node consisting of resources, having the same policy as the gcd node, in its set. If we assign distinct values from $1 - m$ termed as *key values*, to each distinct rule in R , then the subsets $S_1, S_2, \dots, S_m$ can be easily created using Bucket sort as follows: we create m buckets labelled from $1 - m$, for each distinct rule. As we traverse through the rules in R , we simply evaluate its label and place it in the bucket having the same label.
- After which, it computes the $w_{\max}$ weight - the maximum weight over the weights of all nodes in F , and computes the set T consisting of all nodes with $w_{\max}$ weight and removes all these nodes from F .
- Then the nodes in set T are added to G using the ADD-NODESET-TO-G procedure.
- The above two steps are repeated till $F = \emptyset$; that is, the above two steps are repeatedly executed to (possibly) add all the nodes in F to G .

```

procedure ADD-NODESET-TO-G( $G, F, T, w_{\max}$ )  $\triangleright G = (V; E; r)$ 
  if  $w_{\max} = r.\text{weight}$  then  $\triangleright$  Attach resources having same weight as  $r$  to  $r$ 
     $\text{ATTACH-RESOURCES-TO-R}(G, T)$ 
  else
    for each  $s \in T$  do  $\triangleright$  Add permanent nodes in  $T$  to  $G$ 
      if  $s.\text{temp} = \text{false}$  then
         $\text{ADD-NODE-TO-G}(G, s, \emptyset, \text{false})$ 
      end if
    end for
    for each  $s \in T$  do  $\triangleright$  Check child nodes of  $r$  to add  $gcd$  node
       $cset \leftarrow \emptyset$ 
       $cset \leftarrow \text{CURR-MAX-WEIGHT-SET}(G, s)$ 
      if  $cset \neq \emptyset$  then
         $gcd = \text{CREATE-NODE}(cset)$ 
        if  $!s.\text{temp}$  then
          if  $(s.\text{weight} = gcd.\text{weight})$  then
             $\text{ADD-NODE-TO-G}(G, gcd, cset, \text{true})$ 
          else  $\triangleright$  Mark  $gcd$  as a temporary node
             $gcd.\text{temp} \leftarrow \text{true}$ ;  $gcd.\text{set} \leftarrow s$ 
             $gcd.\text{size} \leftarrow 1$ 
             $F \leftarrow F \cup \{gcd\}$ 
          end if
        else
          if  $gcd.\text{weight} = s.\text{weight}$  then
             $\text{ADD-NODE-TO-G}(G, gcd, cset, \text{true})$ 
          end if
        end if
      end for
    end for
  end if

```

```

    end if
  end for
end if
end procedure

```

The ADD-NODESET-TO-G procedure adds the set of nodes in T to G . The outline of this procedure is as follows:

- If $w_{\max} = r.weight$, the ADD-NODESET-TO-G procedure directly attaches to r all the resources in every non-temporary node $s \in T$ and exits. Otherwise, it implements the below steps.
- The procedure loops through each node in T to (possibly) add it to G . If s is not a temporary node (indicated by the *temp* node attribute), it is added to G as a child of the root r .
- Then every child of r is evaluated to compute *cset* - the set consisting of all nodes including s (or including $s.set$, if s is a temporary node). *cset* is computed by the CURR-MAX-WEIGHT-SET function.
- A new *gcd* node *gcd* is computed for the set of resources in *cset*, if *cset* $\neq \emptyset$. If s is a permanent node, then it checks if $gcd.weight = s.weight$. If it is, *gcd* is added to G ; otherwise *gcd* is tagged as “temporary” – to be considered for evaluation later – and added to F . If s is a temporary node and if the node is in *s.set* is still part of the segment formed by *gcd*, then *gcd* is added to G . Otherwise, it is automatically removed from F , as all the nodes in T are removed from F , before adding them to G in Algorithm 2.

Recall that nodes are added in batches (T) to G in decreasing order of their weights. Therefore, in the ADD-NODESET-TO-G procedure when $gcd.weight < s.weight$, the node *gcd* is not added to G , but instead added to F (if it is not a temporary node), to be evaluated when the set of nodes (T) having weight = $gcd.weight$ is processed. Furthermore, temporary nodes are associated with a node for which they were created. For example, when a permanent node s from T is added to G , we check to see if any child nodes of r can form a segment with s . If the weight of this segment is less than the weight of s , we create a temporary node for the segment and evaluate it later when the nodes with its weight are being evaluated.

During the execution of the ADD-NODESET-TO-G function, while evaluating a set $s \in T$, let the root r have $c_1, c_2, \dots, c_k$ child nodes. Let the GCD of the policies of $(c_1, s), (c_2, s), \dots, (c_k, s)$ be $g_1, g_2, \dots, g_k$. Let *cmax* be the maximum weight over all the weights of the policies $g_1, g_2, \dots, g_k$. Then the primary objective of the CURR-MAX-WEIGHT-SET function is to compute the set (not necessarily the largest) of all child nodes $snew = \{c_{i_1}, c_{i_2}, \dots, c_{i_l}\}$, such that $snew \subseteq \{c_1, c_2, \dots, c_k\}$, and $gcd(c_{i_1}, c_{i_2}, \dots, c_{i_l}, s)$ has *cmax* weight.

To compute *snew*, CURR-MAX-WEIGHT-SET function maintains a stack S (this can also be achieved with an array/list). Initially, S is empty and *cmax* = 0. While evaluating the child nodes of r , S contains the nodes having the current/local maximum weight. Therefore, by the end of the **for** loop, S contains all the child nodes, such that the weight of the GCD of their policies is *cmax*. *snew* contains the largest possible set of nodes if *cmax* = $s.weight$. However, if *cmax* $\neq s.weight$, it does not necessarily store the largest set. This is not a problem, as a node corresponding to *cmax* is added as a temporary node (if s is

a permanent node) to F , in which case it is evaluated later and the maximum possible set of nodes corresponding to it, is formed at that time.

Note that, we do not consider s while computing this set, because $gcd(s, s) \geq gcd(c_i, s)$, where $1 \leq i \leq k$. Hence, if s is included in the set, *cmax* = $s.weight$. This would be a problem if $snew = \{s\}$, as we cannot form a segment with only one element. Therefore, we compute S without considering s and if $S \neq \emptyset$ we add s to it. Furthermore, if *cmax* is equal to the weight of the root, it is pointless to create another sub segment with its weight = $r.weight$. Therefore, we disregard this case.

function CURR-MAX-WEIGHT-SET(G, s)

```

 $S \leftarrow \emptyset$ ;  $snew \leftarrow \emptyset$ 
 $Sgcd \leftarrow \emptyset$   $\triangleright$  gcd of policies of the nodes in  $S$ 
 $cmax \leftarrow 0$   $\triangleright$  Current Max.weight of gcd of policies
 $flag \leftarrow false$ 
for each child  $c$  of  $r$  do
  if  $s.temp \wedge c \in s.set$  then
     $flag \leftarrow true$ 
  end if
  if  $(!s.temp \wedge c \neq s) \vee (s.temp \wedge c \notin s.set)$  then
     $tempp \leftarrow gcd(c.p, s.p)$ 
     $tempw \leftarrow weight$  of  $tempp$ 
    if  $cmax < tempw$  then
      Empty  $S$ ; Push( $S, c$ )
       $Sgcd \leftarrow gcd(s, c)$ 
       $cmax \leftarrow tempw$ 
    else if  $cmax = tempw$  then
       $Sgcd \leftarrow gcd(Sgcd, c)$ 
       $Sgcdweight \leftarrow weight$  of  $Sgcd$ 
      if  $Sgcdweight = cmax$  then
        Push( $S, c$ )
      end if
    end if
  end if
end for
if  $!S.empty \wedge cmax \neq r.weight \wedge s.temp \wedge flag$  then
  Push( $S, s.set$ )
  while  $S \neq \emptyset$  do
     $snew \leftarrow snew \cup Pop(S)$ 
  end while
end if
if  $!S.empty \wedge cmax \neq r.weight \wedge !s.temp$  then
  Push( $S, s$ )
  while  $S \neq \emptyset$  do
     $snew \leftarrow snew \cup Pop(S)$ 
  end while
end if
return  $snew$ 
end function

```

The ATTACH-RESOURCES-TO-R procedure attaches resources having the same weight as r to r .

procedure ATTACH-RESOURCES-TO-R(G, T) $\triangleright G = (V; E; r)$

```

for each  $s \in T \wedge !s.temp$  do
  if  $!s.isresource$  then
    for each  $res \in s.set$  do
       $res = CREATE-NODE(res)$ 
      ADD-NODE-TO-G( $G, res, \emptyset, false$ )
    end for
  end if
end for

```

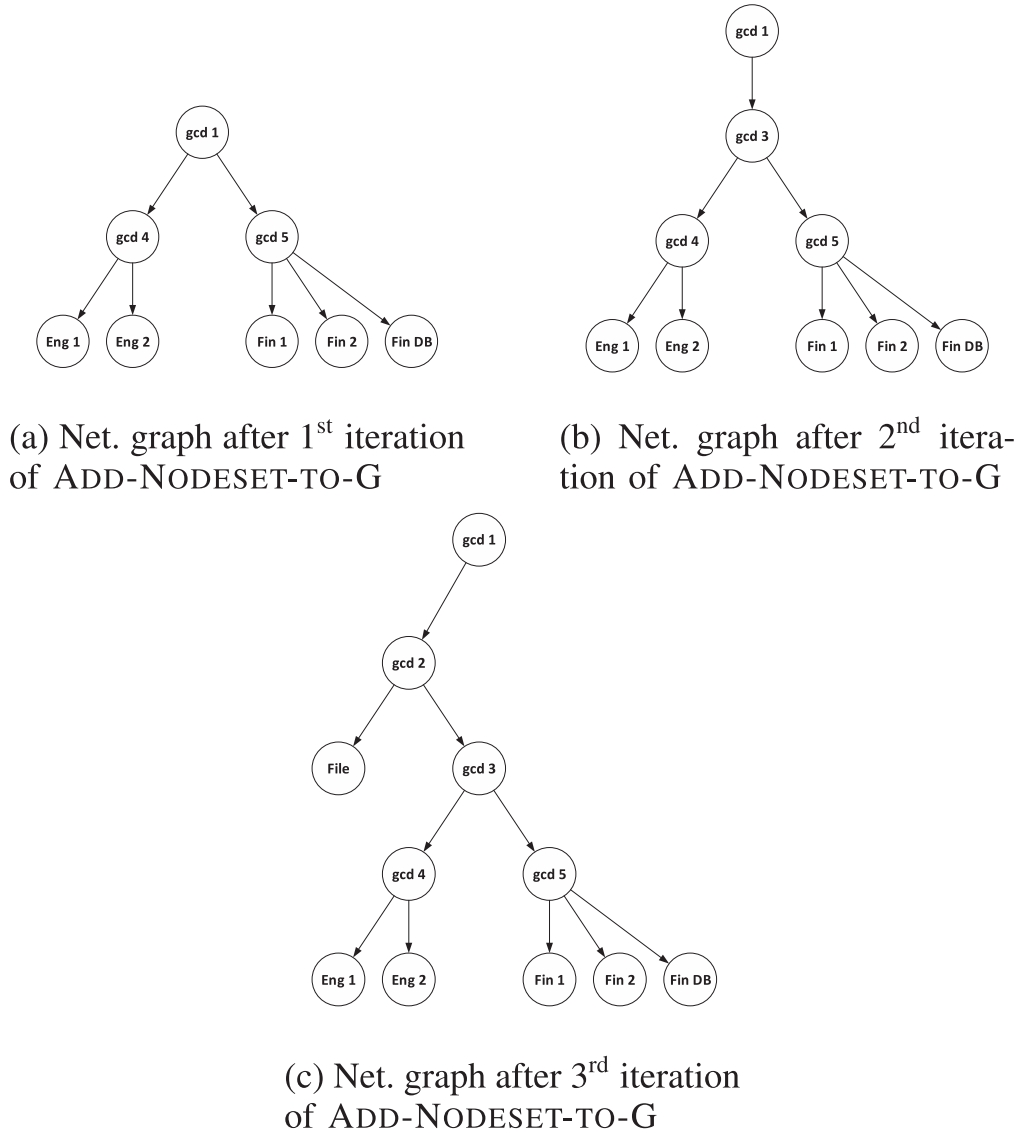


Fig. 10 – Progression of RNS for the illustrative example.

```

else
  ADD-NODE-TO-G( $G, s, \emptyset, false$ )
end if
end for
end procedure

```

We present the auxiliary CREATE-NODE function, and the ADD-NODE-TO-G procedure in [Appendix C](#), for clarity and completeness.

[Fig. 10](#) shows the progression of executing the RNS algorithm on the illustrative example. The last iteration generates the network shown in [Fig. 15](#).

RNS Algorithm: Robustness and complexity

Theorem 2. *The RNS Algorithm constructs a robust network graph.*

Proof. The proof is by contradiction. We assume that at least one of the three criteria required for a network graph to be robust ([Section 5](#)), is not satisfied.

Suppose Condition 1 is not satisfied. Then there exists at least one path in G from the root to the parent of a resource, such that the SDD strategy is not satisfied. In the RNS algorithm, the root is added first to an empty graph G . After which nodes both resource and gcd nodes are added in non-increasing order of their weights to G by first attaching them to the root only in the ADD-NODESET-TO-G procedure. In ADD-NODESET-TO-G, when a gcd node gcd is added to the root r , $cset$ contains all its child nodes. The policy at gcd is equal to the GCD of policies of all nodes in $cset$. Furthermore, while adding child nodes from $cset$ to gcd , we check if any child node's weight is equal to $gcd.weight$, in which case we delete the child node (if its not a resource) and attach all its children to gcd . Therefore, in all the cases the SDD strategy is satisfied while adding resources and gcd nodes. Therefore, the SDD strategy is satisfied in all paths from the root to the node of the parent of a resource, which is a contraction.

Suppose Condition 2 is not satisfied. Then, the condition stated under Definition 3 is not satisfied; that is, there exists a segment S , such that a resource $r \in S$ and a resource $r' \notin S$ have commonalities with more weight than the weight of the commonalities between elements of the segment S .

Without loss of generality, let us assume that $w(p(r)) \leq w(p(r'))$. Nodes are added to G in non-increasing order of their weights. Therefore, when r is added to G , all the existing nodes in G have weight greater than or equal to $r.weight$. Since $r.weight \leq r'.weight$, G must include r' . Furthermore, while evaluating clusters of resources (having weight greater than or equal to $r.weight$) including r , we choose a cluster having the maximum weight (and possibly the maximum size) among all such clusters evaluated, as the segment S . Therefore, it must be the case that a cluster having r and r' is evaluated. Since r and r' have commonalities with higher weight, then the chosen cluster forming a segment must contain r and r' . Therefore, the segment S must contain both r, r' , which is a contradiction.

Suppose Condition 3 is not satisfied; that is, superfluous firewall chaining exists in G . Then there exists at least one firewall/ gcd node with a single node that is a firewall attached. Recall that gcd nodes are added to G only when more than one node can be attached to them. Therefore, by the end of the RNS algorithm, G has no superfluous firewalls, which is a contradiction. $\square$

Theorem 3. Let $R = \{r_1, r_2, \dots, r_n\}$ be the input of size n to the RNS Algorithm. Then, the running time of RNS is $O(n^2)$.

Proof. Using numeral key values between $1 - m$ as labels for distinct rules in R , and Bucket sort, the time required to compute the subsets $S_1, S_2, \dots, S_m$, where $m \leq n$ is $O(n)$.

Since gcd nodes (either permanent or temporary) are created for each set $s \in S_1, S_2, \dots, S_m$ exactly once, the number of gcd nodes in the network graph is at most n . Analogously, the number of temporary nodes created during the execution of the RNS algorithm is also at most n . Hence the total time required for the RNS algorithm is $O(n)$, plus the total time required to execute the ADD-NODESET-TO-G procedure.

Let $t \leq n$ be the number of distinct weights of policies in the resource set R . Then, the ADD-NODESET-TO-G procedure is executed t times, and the total number of nodes processed by it is at most $2n$ (since $|F| \leq 2n$). Hence the time taken by ADD-NODESET-TO-G is $2n$ plus the time taken by the CURR-MAX-WEIGHT-SET for each node s in F where $s.weight > r.weight$.

The total time required by the CURR-MAX-WEIGHT-SET procedure is equal to the number of child nodes of r the procedure evaluates to construct the $snew$ set for all nodes in F . The maximum number of such nodes is at most $2n$ (total number of nodes in G) at each execution of the CURR-MAX-WEIGHT-SET procedure. Further, the CURR-MAX-WEIGHT-SET procedure is executed at most $2n$ times (number of nodes in F). Hence, the total time required by the CURR-MAX-WEIGHT-SET procedure is at most $O(n^2)$.

Therefore the total running time for the RNS Algorithm is $O(n^2)$. $\square$

7. Application of RNS algorithm in SDN

OpenFlow (McKeown et al., 2008) protocol has emerged as an approach to remotely program switches while having a global view. When a switch has no entries in its flow tables to handle a packet, it forwards it to a controller which instructs the switch on how to handle such packets. This approach of decoupling control of traffic from the data plane is referred to as SDN (Feamster et al., 2014; Nunes et al., 2014). SDN can be characterised by the separation of the control plane from the data plane in a network: the physical movement of data (packet forwarding) in a network is separated from the logic and control governing this movement. Hence, the decoupling leads to a separation of concerns, which simplifies the network control design and its maintenance.

In a typical SDN architecture, a central controller residing in the control plane governs the movement of data while the switches residing in the data plane strictly forward packets as directed by the controller. In the literature, we find the following architectures for implementing firewalls in the SDN:

- Single centralised firewall:* In Javid et al. (2014); Kumar and Srinath (2016); Suh et al. (2014) the authors propose an architecture where the entire network is governed by a stateless centralised firewall at the control plane. In this architecture, the policy resides at the firewall and switches forward all traffic to the controller which checks the policy and instructs the switch to drop or forward the packets.
- Distributed firewalls at control plane:* In Kaur et al. (2015); Morzhov et al. (2016); Pena and Yu (2014); Tran and Ahn (2015) the authors propose an architecture with distributed firewall at the control plane. The firewalls implemented in this architecture are stateless firewalls. Each switch is assigned a firewall at the controller. The firewall reads the policy and inserts deny rules into the flow table of the switch. The denied traffic is then dropped by the switch, and all other traffic is forwarded to the controller which instructs the switch on how to forward the packet. The proposed architectures in Tran and Ahn (2016) and Gupta et al. (2016) are stateful firewalls that follow a similar process.
- Distributed firewalls at the data plane:* In Caprolu et al. (2019); Chowdhary et al. (2018) the authors propose an architecture that transforms the data plane switches into stateful firewalls. These approaches take advantage of stateful data plane technologies to allow switches to act as stateful firewalls without the need to forward packets to the controller.

As we shall explain in the following section, the RNS algorithm is used to generate the network topology and specific firewall policies for each segment that we put under a switch in the data plane (see Fig. 11). In this section we adopt a distributed stateful firewall architecture that combines Architecture (b) with stateful firewall. Our approach is illustrated in Fig. 12.

7.1. Use of RNS in SDN

We start by running the RNS algorithm on the set of resources of a network and their policies to obtain a secure and robust

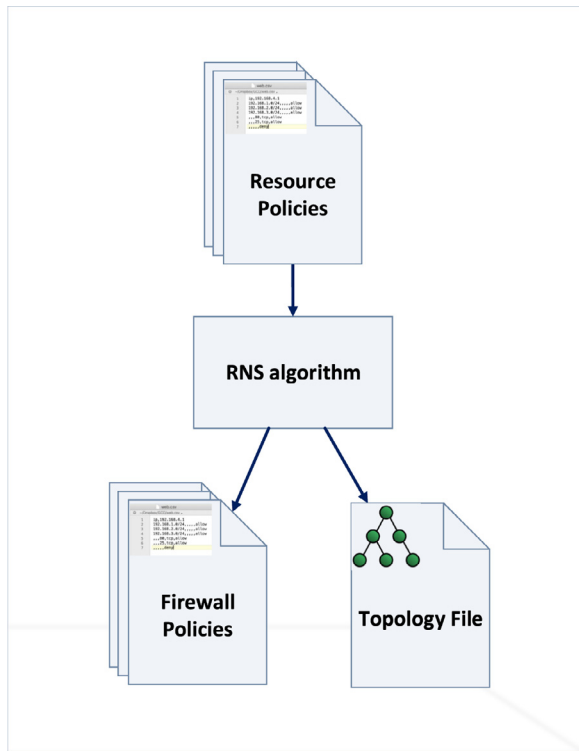


Fig. 11 – RNS algorithm.

network topology and the policies of its firewalls. The design process involves two stages. In the first stage, the policies of the resources as well as a predefined weight function are fed as input to the RNS algorithm.

In the second stage, we use *mininet* (an SDN emulation tool) (de Oliveira et al., 2014) and POX controller (Al-Shabibi and McCauley, 2020) to create and simulate the network corresponding to the topology obtained from the first stage. In the SDN controller to handle each switch in the network we create a stateful firewall object. Each firewall object has a state table, a module to communicate with the actual switch, and a package consisting of modules to handle (referred to as handlers) different protocols such as TCP, UDP, or ICMP. Each firewall object fetches its policy, generated by the RNS algorithm, and stores it (See Fig. 12).

We use the network of the example given in Section 2 to illustrate the stages of designing SDN based on the RNS algorithm. We obtain the topology given in Fig. 12. To assess whether the traffic in the network is governed according to the policies set by the RNS algorithm, we run a reachability test as shown in Fig. 13. The reachability test is performed using the command *pingall*, which allows every data plane host to ping every other host. In Fig. 13, the first line of the system response to *pingall* command starts with *eng2*, which is the name of the resource initiating the ping request. Then we have the arrow *->* followed by resource names such as *eng2*, *file*, *email*, and *web* indicating a successful communication of *eng2* with these resources. An *x* indicates that the initiating host is failing to establish a communication with the target host.

8. Other applications of RNS algorithm

The RNS algorithm can be applied to build computer networks in several conceivable scenarios. Here we highlight some of the scenarios where traditional networks fail, but the RNS algorithm is applicable.

8.1. Internet of things

IoT (e.g., Madakam et al., 2015) is a vision for developments that would touch many aspects of our daily lives. At its core IoT aims at having heterogeneous physical objects, for example sensors, devices, tools, machines, wearable items and software components, connect and communicate through the Internet, forming several cooperating eco-systems delivering a global activity. Many organizations all over the world are initiating IoT driven ventures to improve their business suppleness or to better meet the demands of their customers. Networking a large number of elements of IoT, while ensuring the security of each element is one of the many challenges (Ammar et al., 2018; Conti et al., 2018). Many existing models (e.g., Ammar, Russello, Crispo, 2018) require devices to be resources which rely on the manufacturer's cloud-based servers to function. The manufacturer's cloud clusters these resources into network segments to enable them to securely communicate with other resources within or outside the cloud. The RNS algorithm would play a significant role in achieving an automatic and systematic segmentation of the manufacturer's networks. Moreover, our proposed approach for segmentation builds a secure network by default.

8.2. On-the-fly-networks/dynamic networks:

In a secure dynamic networking environment, it is vital that the reasoning required to cope with the changing security needs, and communicating the attained security decisions to the concerned access control nodes is automated. Moreover, this automation is critical for large networks with a considerable number of dynamic resources. The RNS algorithm, generates an on-the-fly best segmentation, and policy-based protection to the network's components, while ensuring consistency among the resource policies.

For example in an SDN setting, the RNS algorithm is re-executed to generate an updated topology. The updated topology is compared to the old topology to decide on which hosts, switches, links to be added or deleted. To implement this, the differences are written in a python script that executes *mininet* Command-line interface (CLI) to carry out the changes. The topology is then updated on-the-fly by running the script on the *mininet* CLI. After which the controller is notified of the changes, which then creates new firewalls and updates policies of affected ones accordingly.

Further, in real networks a resource might be accessed from several entry points or used from other networks. Therefore, in dynamic configuration of networks, special care needs to be taken in handling such resources. If a resource is deemed out of service in one network, it should not be automatically remove without considering its usage from outside of the network under consideration. In our paper (Alabbad et al., 2020),

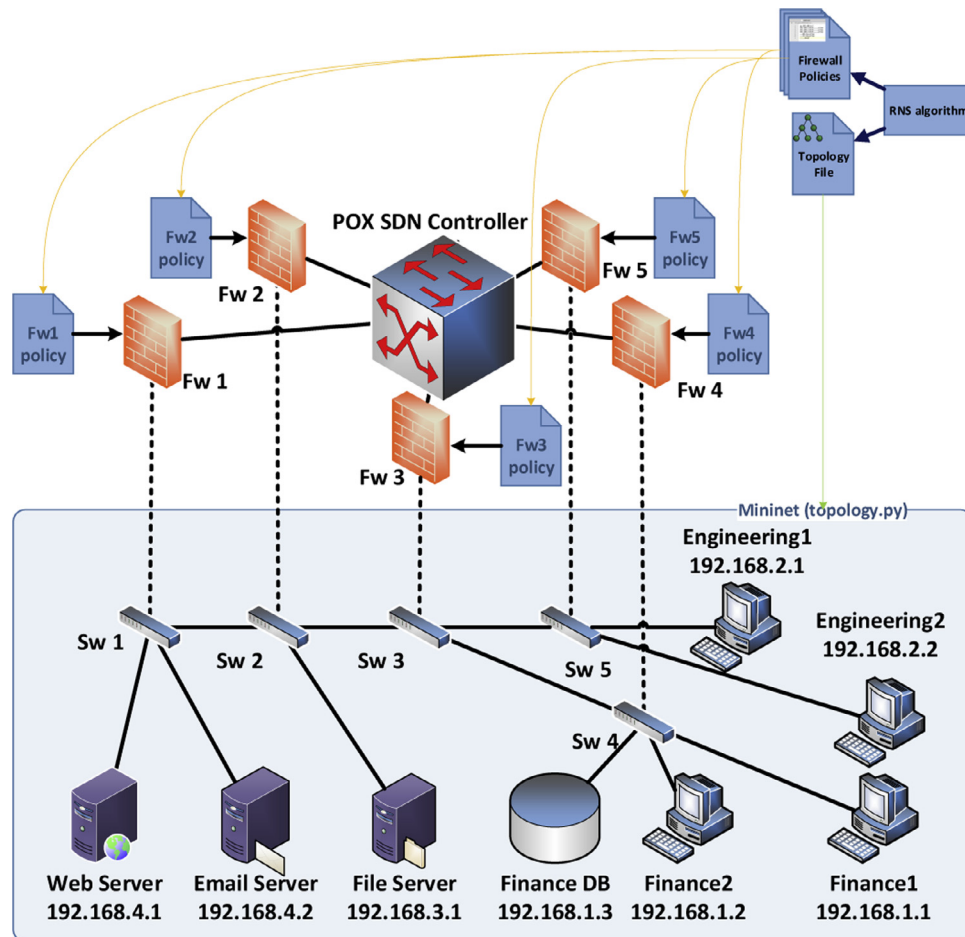


Fig. 12 – SDN simulation.

```

mininet> pingall
*** Ping: testing ping reachability
eng2 -> eng1 X X X file email web
eng1 -> eng2 X X X file email web
fin1 -> X X finDB fin2 file email web
finDB -> X X fin1 fin2 file email web
fin2 -> X X fin1 finDB file email web
file -> X X X X X email web
email -> X X X X X web
web -> X X X X X email
*** Results: 51% dropped (27/56 received)
mininet>

```

Fig. 13 – Reachability Test of the illustrative example Resources.

we discuss the usage of the RNS algorithm to segment networks having several entry points.

9. Discussion

We proposed two algorithms, Exp-RNS and RNS, for building a robust network topology. The Exp-RNS algorithm has an expo-

nential running time, and RNS has a polynomial running time. Despite its exponential complexity, we presented the Exp-RNS algorithm for its simplicity in demonstrating the usage of the formalism in building a robust network. We later introduced RNS, which can easily meet the needs of the most of real-time network configuration in dynamic networks.

The restriction in the superfluous firewall chaining concept mentioned in Section 5 can be further tightened, where a firewall having no resources attached to it (irrespective of the number of firewalls attached to it) is removed from the network. This would not only simplify the network, but also significantly economize it by minimizing the number of firewalls required to produce a secure and robust network. The Exp-RNS and RNS algorithms can be easily modified with minor changes to adapt to the tighter version of firewall chaining. Both these versions have its advantages and drawbacks. The superfluous firewall chaining mentioned in Section 5 would result in a network having more firewalls and more number of layers protecting the resources, however building such a network could be expensive. The tighter version, results in a network having less firewalls and fewer layers protecting the resources; however, it is economical.

Our approach is for designing and building networks, and is not focused on improving the security in existing networks. Quantified attack graphs (Albanese et al., 2012; Dewri et al.,

2007; Ingols et al., 2006; M.J. Almohri et al., 2016; Poolsappa-sit et al., 2012; Wang et al., 2006), and similar formalisms (e.g., Rahman and Al-Shaer, 2013) have been proposed to harden the security of existing networks. However, our approach can be used to assess the security robustness of an existing network in the following ways. One way is to use the GCD to calculate the best policy for each of the segments of the existing networks. Then, we compare the calculated firewall policy for each segment to the existing policy. This would reveal any missing rules or any flaws in the policy of the existing firewall. The second way is to verify whether the existing network implements DD strategy. A third way is to use RNS to get the appropriate robust network for the resources that are in the existing network. Then, we compare the topologies (existing and calculated) with regard to their effectiveness in protecting the resources. This comparison might lead the network security officer to better tune the existing network for an enhanced security.

Today's networks are very large in terms of the number of their nodes. The abstraction adopted in the family approach allows to collapse a subset of nodes into one node that is protected by a family of policies obtained from the policies of the abstracted nodes. This approach for example allows us to focus on the design of a network in a geographic area while considering the rest of the network in another geographic area as a node running a family of policies. It is a direct usage of the principle of divide and conquer.

It is also important to put our results in perspective with Network Address Translation (NAT) which is a translation mechanism executed by a firewall that is in contact with the Internet outside of a private network. It assigns public addresses to a computer inside a network. Its purpose is to limit the number of public addresses that internal nodes within a network need to know about external resources. In our work, we assume that only the node that we call root will have this translation capabilities. Our purpose is to design local networks and NAT can be placed on the boundary between the local network to be designed and the external Internet (i.e., on the entry points (roots) of the network). Hence, our policies use only local addresses to the designed network.

We executed the RNS algorithm on a network with 100 resources, on an *Apple MacBook Pro* with a CPU 2.7 GHz Intel Core i5 and a memory of 8 GB 1867 MHz DDR3. The algorithm took 24.40 seconds to calculate the network topology and the policies to be enforced at its firewalls. Then, we inactivated ten resources out of the previous 100 resources, and executed the RNS algorithm to recalculate the new network topology and the policies of its firewalls. The machine took 24.36 seconds. While the above performance numbers do not constitute, by any stretch an empirical assessment – a definitive real performance indicator, they give an idea about the range of time needed. Although the two numbers do not differ much, we want to emphasize that in both these cases; that is, with 100 resources and 90 resources, we re-calculate the whole network topology. Many targeted strategies can be conceived by considering the characteristics of the resources that are added or removed. For instance, when a resource is added/removed, if this change affects only a subnetwork, we could re-calculate the new topology only for the subnetwork affected by this change. This would require running the RNS algorithm for a

small number of resources that are part of the subnetwork as input. In this situation, we will see enhanced performance which is appropriate for real-time network reconfiguration. In many situations, however, the performance of the RNS algorithm when used to recalculate the whole topology of the network for each network change is acceptable to reconfigure the network, for example, re-configuring the network in response to several security threats. Therefore, a comprehensive empirical study to further assess this aspect is needed. Moreover, we also need further research related to the usage of RNS algorithm in many networks such as mesh networks.

10. Conclusion and future work

In this paper, we formally define network segmentation. Then, we propose novel algorithms to build a robust network architecture which segments resources such that maximum access protection is provided to its resources. Our formalism is based on the mathematical framework of PFA, where a policy is treated as a product and the sets of policies governing a firewall are equivalent to product families, and an atomic rule in a policy is equivalent to a feature, and are modelled as a guarded command. Both the Exp-RNS and RNS algorithms use the gcd and the refinement relations to segment resources and build a robust network.

The advantage of building a robust network using the RNS algorithm is that public facing resources are placed close to the edge of the network and have limited access to internal resources. Resources requiring more security are placed farther from the edge of the network and protected by many layers. Furthermore, access from one segment to the other is controlled by internal firewalls. This is a common practice in network segmentation that is driven by intuition. We give the mathematical rationale for it. Moreover, the advantage of having a formal treatment of the problem allows for automated solutions enabling handling of huge networks.

The RNS algorithm could be scaled to enforce additional requirements/constraints on the segmentation to achieve the desired protection. For example, an organization can limit the number of levels required. Furthermore, additional measures can be added to achieve the desired segmentation, for example risk assessment measures (Wagner et al., 2016) such as: infection spread ratio, and cleansing time. A future research work could incorporate these additional factors in the weight function discussed in 5.

We note that a huge body of work focusing on experimental evaluation in several contexts; for example evaluating the RNS algorithm in dynamic networks and IoT would be another important area of research. Furthermore, based on the evaluations in different settings, one could propose improvements to the algorithm to better meet the needs of some settings.

Funding

This study was funded by [Natural Sciences and Engineering Research Council of Canada](#) – NSERC – (CA)SRGPIN-6115-2014).

Ethical approval:

This article does not contain any studies with human participants or animals performed by any of the authors.

Declaration of Competing Interest

All authors declare that they have no conflict of interest.

Appendix A

A1. Proofs of lemmas.

A1.1. Proof of Lemma 1.

$$\begin{aligned}
 & \neg(p(r) \neq 0_{\mathcal{F}} \wedge (\forall a, b \mid (a, b) \in E \cdot p(b) \sqsubset_{\mathcal{F}} p(a))) \\
 \iff & \quad (\text{De Morgan Law}) \\
 & (p(r) = 0_{\mathcal{F}}) \vee (\exists a, b \mid (a, b) \in E \cdot p(b) \not\sqsubset_{\mathcal{F}} p(a)) \\
 \implies & \quad (\text{Strengthening}) \\
 & (\exists a, b \mid (a, b) \in E \cdot p(b) \not\sqsubset_{\mathcal{F}} p(a)) \\
 \Leftarrow & \quad (\neg(v, l) \in E) \\
 & p(l) \not\sqsubset_{\mathcal{F}} p(v) \\
 \iff & \quad (\neg p(v) = \gcd(p(v), p(l))) \\
 & p(l) \not\sqsubset_{\mathcal{F}} \gcd(p(v), p(l)) \\
 \iff & \quad (\text{From the hypothesis } p(l) = 1_{\mathcal{F}}) \\
 & 1_{\mathcal{F}} \not\sqsubset_{\mathcal{F}} \gcd(p(v), 1_{\mathcal{F}}) \\
 \iff & \quad (\text{Because } \gcd(p(v), 1_{\mathcal{F}}) = 1_{\mathcal{F}}, v \text{ is parent of } l) \\
 & 1_{\mathcal{F}} \not\sqsubset_{\mathcal{F}} 1_{\mathcal{F}} \\
 \Leftarrow & \quad (\text{From the definition of } \sqsubset_{\mathcal{F}}) \\
 & \text{true}
 \end{aligned}$$

□

A1.2. Proof of Lemma 2.

$$\begin{aligned}
 & \neg(p(r) \neq 0_{\mathcal{F}} \wedge (\forall a, b \mid (a, b) \in E \cdot p(b) \sqsubset_{\mathcal{F}} p(a))) \\
 \iff & \quad (\text{De Morgan Law}) \\
 & (p(r) = 0_{\mathcal{F}}) \vee (\exists a, b \mid (a, b) \in E \cdot p(b) \not\sqsubset_{\mathcal{F}} p(a)) \\
 \implies & \quad (\text{Strengthening}) \\
 & (\exists a, b \mid (a, b) \in E \cdot p(b) \not\sqsubset_{\mathcal{F}} p(a)) \\
 \Leftarrow & \quad (\neg(r, s) \in E) \\
 & p(s) \not\sqsubset_{\mathcal{F}} p(r) \\
 \iff & \quad (\neg p(s) = \gcd(p(s), \gcd(p(l_1), p(l_2))) \text{ and } p(r) = \\
 & \quad \gcd(p(r), p(s)) \text{ and from the hypothesis } \\
 & \quad \gcd(l_1, l_2) = 1_{\mathcal{F}}) \\
 & 1_{\mathcal{F}} \not\sqsubset_{\mathcal{F}} 1_{\mathcal{F}} \\
 \Leftarrow & \quad (\text{From the definition of } \sqsubset_{\mathcal{F}}) \\
 & \text{true}
 \end{aligned}$$

□

A1.3. Proof of Lemma 3.

□

$$\begin{aligned}
 & \neg(p(r) \neq 0_{\mathcal{F}} \wedge (\forall a, b \mid (a, b) \in E \cdot p(b) \sqsubset_{\mathcal{F}} p(a))) \\
 \iff & \quad (\text{De Morgan Law}) \\
 & p(r) = 0_{\mathcal{F}} \vee (\exists a, b \mid (a, b) \in E \cdot p(b) \not\sqsubset_{\mathcal{F}} p(a)) \\
 \implies & \quad (\text{Strengthening}) \\
 & (\exists a, b \mid (a, b) \in E \cdot p(b) \not\sqsubset_{\mathcal{F}} p(a)) \\
 \Leftarrow & \quad (\neg(u, v) \in E) \\
 & p(v) \not\sqsubset_{\mathcal{F}} p(u) \\
 \iff & \quad (\text{From hypothesis } p(u) = (\gcd v_i \mid (v, v_i) \in \\
 & \quad E \cdot p(v_i)) = p(v)) \\
 & p(v) \not\sqsubset_{\mathcal{F}} p(v) \\
 \Leftarrow & \quad (\text{From the definition of } \sqsubset_{\mathcal{F}}) \\
 & \text{true}
 \end{aligned}$$

A1.4. Proof of Lemma 4(a).

$$\begin{aligned}
 & S \text{ is a segment} \\
 \iff & \quad (\text{Definition 3}) \\
 & (\forall r, r' \mid r \in S \wedge r' \in (R - S) \cdot w_P(\gcd(p(r), p(r'))) \leq w_P(\gcd(r \mid \\
 & \quad r \in S \cdot p(r))) \\
 \iff & \quad (\neg S \text{ is a set consisting of resources having co-} \\
 & \quad \text{prime policies, gcd of co-prime policies} = 1_{\mathcal{F}}) \\
 & (\forall r, r' \mid r \in S \wedge r' \in (R - S) \cdot w_P(\gcd(p(r), p(r'))) \leq w_P(1_{\mathcal{F}})) \\
 \Leftarrow & \quad (\text{From hypothesis } p(r') = 1_{\mathcal{F}}) \\
 & (\forall r, r' \mid r \in S \wedge r' \in (R - S) \cdot w_P(\gcd(p(r), 1_{\mathcal{F}})) \leq w_P(1_{\mathcal{F}})) \\
 \iff & \quad (\text{Because } (\forall a \mid \neg \gcd(a, 1_{\mathcal{F}}) = 1_{\mathcal{F}})) \\
 & (\forall r, r' \mid r \in S \wedge r' \in (R - S) \cdot w_P(1_{\mathcal{F}}) \leq w_P(1_{\mathcal{F}})) \\
 \Leftarrow & \quad (\text{reflexivity of } \leq) \\
 & \text{true}
 \end{aligned}$$

□

A1.5. Proof of Lemma 4(b).

$$\begin{aligned}
 & S \text{ is a segment} \\
 \iff & \quad (\text{Definition 3}) \\
 & (\forall r, r' \mid r \in S \wedge r' \in (R - S) \cdot \\
 & \quad w_P(\gcd(p(r), p(r'))) \leq w_P(\gcd(r \mid r \in S \cdot p(r))) \\
 \Leftarrow & \quad (\text{From hypothesis: } (\forall r, r' \mid r \in S \wedge r' \in (R - \\
 & \quad S) \cdot \gcd(p(r), p(r')) = 1_{\mathcal{F}})) \\
 & (\forall r, r' \mid r \in S \wedge r' \in (R - S) \cdot w_P(1_{\mathcal{F}}) \leq w_P(\gcd(r \mid r \in \\
 & \quad S \cdot p(r))) \\
 \Leftarrow & \quad (\neg r \in S \text{ and } p(r) = 1_{\mathcal{F}}, \text{ which makes } \gcd(r \mid \\
 & \quad r \in S \cdot p(r)) = 1_{\mathcal{F}}) \\
 & (\forall r, r' \mid r \in S \wedge r' \in (R - S) \cdot w_P(1_{\mathcal{F}}) \leq w_P(1_{\mathcal{F}})) \\
 \Leftarrow & \quad (\text{reflexivity of } \leq) \\
 & \text{true}
 \end{aligned}$$

□

Appendix B. Exp-RNS Algorithm Functions

In this section we present detailed algorithms for the five main tasks listed in the Exp-RNS algorithm (see [Section 6.1](#)).

B1. COMPUTE-GCD function

Here we explain the COMPUTE-GCD function in detail. The function first computes the power set of the set of resources R , $\mathcal{P}(R)$. It then computes the GCD for each set in $\mathcal{P}(R)$. While computing the GCDs, the function maintains a set **GCD**, consisting of objects called the *gcd* objects. Each *gcd* object has the following attributes: *p*, *set*, *weight*, and *size*. Attribute *set* represents the set of resources, which is a set in $\mathcal{P}(R)$. Attribute *p* represents the GCD of the policies of the resources in *set*. Attribute *weight* is the weight of the policy *p*, and attribute *size* represents the cardinality of *set*. To minimize the number of *gcd* objects stored in **GCD**, each object in **GCD** has a unique policy *p*. To achieve this, if the GCD of two sets in $\mathcal{P}(R)$ is the same, then we store the *gcd* object corresponding to the larger set. Further for any two sets $s_1, s_2 \in \mathcal{P}(R)$, if the GCD of policies of s_1 refines the GCD of policies of s_2 , and if both these policies have the same weight, then we only store the *gcd* object corresponding to s_2 .

```

procedure COMPUTE-GCD( $R$ )
   $\mathcal{P}(R) \leftarrow$  power set of  $R$  minus all singleton sets and
  empty set.
  GCD  $\leftarrow \emptyset$   $\triangleright$  initialize GCD set.
  for each  $s \in \mathcal{P}(R)$  do
    Create gcd_s object
     $gcd\_s.p \leftarrow$  Calculate the GCD of the policies of re-
    sources in  $s$ 
     $gcd\_s.weight \leftarrow$  weight of  $gcd\_s.p$ 
     $flag \leftarrow false$ 
    for each  $gcd \in \mathbf{GCD}$  do
      if  $gcd.p = gcd\_s.p$  then
         $flag \leftarrow true$ 
        if  $gcd.size < |s|$  then
           $gcd.size \leftarrow |s|$ 
           $gcd.set \leftarrow s$ 
        end if
      else if  $gcd.p \sqsubset gcd\_s.p \wedge gcd.weight = gcd\_s.weight$ 
then
        GCD = GCD – { $gcd$ }
      else if  $gcd\_s.p \sqsubset gcd.p \wedge gcd.weight = gcd\_s.weight$ 
then
         $flag \leftarrow true$ 
      end if
    end for
    if  $flag = false$  then
       $gcd\_s.set \leftarrow s$ 
       $gcd\_s.size \leftarrow |s|$ 
       $gcd\_s.weight \leftarrow$  weight of  $gcd\_s.p$ 
      GCD  $\leftarrow \mathbf{GCD} \cup \{gcd\_s\}$ 
    end if
  end for
  return GCD
end procedure

```

Table B1 – gcd objects in GCD set, and their corresponding set of resources.

<i>gcd</i> objects in GCD set	Set of Resources
<i>gcd</i> 1	{Web_server, Email_server, File_server, Fin.DB, Fin.1, Fin.2, Eng.1, Eng.2}
<i>gcd</i> 2	{File_server, Fin.DB, Fin.1, Fin.2, Eng.1, Eng.2}
<i>gcd</i> 3	{Eng.1, Eng.2, Fin.DB, Fin.1, Fin.2}
<i>gcd</i> 4	{Eng.1, Eng.2}
<i>gcd</i> 5	{Fin.DB, Fin.1, Fin.2}

[Table B.1](#) gives all the *gcd* objects in the GCD set, and the set of resources in set for the illustrative example after procedure COMPUTE-GCD is executed. In [Appendix D](#), we present the computation of policy for *gcd* 3 (we refer the reader to [\(Khedri et al., 2019, Section 2.2\)](#) for details on the computation of policies for all the *gcd* objects given in [Table B.1](#)).

B2. BUILD-NETWORK-GRAPH function

In this section, we present algorithms for the BUILD-NETWORK-GRAPH function and the ADD-NODESET-TO-G procedure.

```

procedure BUILD-NETWORK-GRAPH( $F, R$ )
   $G \leftarrow \emptyset$   $\triangleright G = (E; V; r)$ 
   $r \leftarrow$  gcd object that has all resources  $R$  in its set attribute
   $r.\pi \leftarrow \emptyset$ ;  $r.resource \leftarrow false$ 
   $V \leftarrow V \cup \{r\}$ 
   $F \leftarrow F$  – minus gcd objects with the same weight as the
  root  $r$ 
  while  $F \neq \emptyset$  do
     $w_{max} \leftarrow$  maximum weight of any  $s \in F$ .
     $T \leftarrow \emptyset$ 
    for each  $s \in F$  do
      if  $s.weight = w_{max}$  then
         $T \leftarrow T \cup s$ ;  $F \leftarrow F - s$ 
      end if
    end for
    ADD-NODESET-TO-G( $G, F, T, w_{max}$ )
  end while
  return  $G$ 
end procedure

```

B2.1. Adding *gcd* nodes in T to the network graph

The *gcd* nodes are added to the network graph based on the weight attribute of the *gcd* object. We add nodes to G in batches (set T) starting with maximum weight to minimum weight. The ADD-NODESET-TO-G procedure adds the set of nodes in T in non-increasing order by their size attribute; that is, nodes in T having the largest size is processed first, followed by a node with next (or same) largest size is processed to add it to G . During the execution of the procedure, at all times we maintain the *resT* set that contains the set of resources for which *gcd* nodes have not been added. Initially, *resT* contains all resources contained in all the *gcd* nodes in T . Then the procedure loops through each node s in T and if $s.set$ still contains resources unaccounted for in G , it is evaluated. During this evaluation, every child c of the root r that refines s is added to $cset$. Then s is added to G by attaching it to r . If $cset$ is not

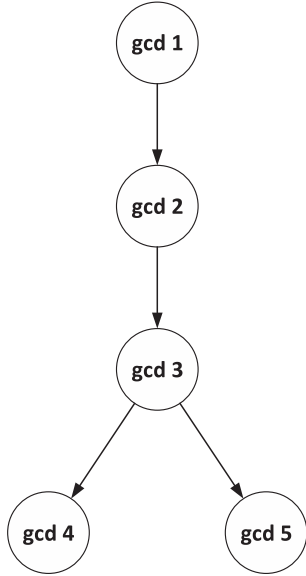


Fig. 14 – The network graph of the illustrative example after BUILD-NETWORK-GRAPH procedure.

empty, then all nodes in it are attached to s as its children, and their connection to r is removed. Finally, the resources in $s.set$ are removed from $resT$ (if they exist).

```

procedure ADD-NODESET-TO-G( $G, F, T, w_{max}$ )  $\triangleright G = (V; E; r)$ 
   $resT \leftarrow \emptyset$ 
  for each  $s \in T$  do
     $resT \leftarrow resT \cup s.set$ 
  end for
   $T \leftarrow T$  nodes ordered in non-increasing order by their
  size attribute.
  for each  $s \in T \wedge (s.set \cap resT \neq \emptyset)$  do
     $cset \leftarrow \emptyset$ 
    for each child  $c$  of  $r$  do
      if  $c.p \sqsubseteq s.p$  then
         $cset \leftarrow cset \cup \{c\}$ 
      end if
    end for
     $V \leftarrow V \cup \{s\}$ 
     $E \leftarrow E \cup \{(r, s)\}$ 
     $s.\pi \leftarrow \{r\}$ 
    if  $cset \neq \emptyset$  then
      for each  $c \in cset$  do
         $E \leftarrow E - \{(r, c)\} \cup \{(s, c)\}$ 
         $c.\pi \leftarrow c.\pi - \{r\} \cup \{s\}$ 
      end for
    end if
     $resT \leftarrow resT - s.set$ 
  end for
end procedure

```

Fig. 14 shows the network graph for the illustrative example produced by the BUILD-NETWORK-GRAPH procedure. Observe that in the graph each path represents a refinement chain (e.g., $gcd4 \sqsubseteq_F gcd3 \sqsubseteq_F gcd2 \sqsubseteq_F gcd1$).

B3. ADD-RESOURCES-TO-NET procedure

In this section, we present an algorithm for the ADD-RESOURCES-TO-NET procedure.

```

procedure ADD-RESOURCES-TO-NET( $G, GCD, R$ )
   $GCD_{max} \leftarrow$  ordered list of  $gcd \in GCD$  in descending order
  by weights.
   $R' = R$ 
  for each  $gcd \in GCD_{max}$  do
    for each  $s \in gcd.set$  do
      if  $s \in R'$  then
         $s.is\_resource \leftarrow true$ 
         $s.\pi \leftarrow s.\pi \cup \{gcd\}$ 
         $V \leftarrow V \cup \{s\}$ 
         $E \leftarrow E \cup \{(gcd, s)\}$ 
         $R' \leftarrow R' - \{s\}$ 
      end if
    end for
  end for
end procedure

```

B4. OPTIMIZE-NETWORK-GRAPH procedure

In this section, we present an algorithm for the OPTIMIZE-NETWORK-GRAPH procedure.

```

procedure OPTIMIZE-NETWORK-GRAPH( $G$ )
   $S \leftarrow \emptyset; T \leftarrow \emptyset$   $\triangleright G = (V; E; r)$ 
   $Push(S, r)$   $\triangleright S, T$  are stack
  while  $S \neq \emptyset$  do
     $u \leftarrow Pop(S)$ 
     $Push(T, u)$ 
    for each  $c$  child of  $u$  do
       $Push(S, c)$ 
    end for
  end while
  while  $T \neq \emptyset$  do
     $u \leftarrow Pop(T)$ 
     $child\_count \leftarrow 0$ 
     $f\_flag \leftarrow false$ 
    for each  $c$  child of  $u$  do
       $child\_count \leftarrow child\_count + 1$ 
      if  $c.is\_resource = false$  then
         $f\_flag \leftarrow true$ 
      end if
    end for
    if  $child\_count = 0 \wedge u.is\_resource = false$  then
       $V \leftarrow V - \{u\}$ 
      for each  $x \in u.\pi$  do
         $E \leftarrow E - \{(x, u)\}$ 
      end for
    else if ( $child\_count = 1 \wedge f\_flag = true$ ) then
       $V \leftarrow V - \{u\}$ 
       $c.\pi \leftarrow c.\pi - \{u\}$ 
       $c.\pi \leftarrow c.\pi \cup \{u.\pi\}$ 
       $E \leftarrow E - \{(u, c)\}$ 
      for each  $x \in u.\pi$  do
         $E \leftarrow E - \{(x, u)\}$ 
         $E \leftarrow E \cup \{(x, c)\}$ 
      end for
    end if
  end while

```

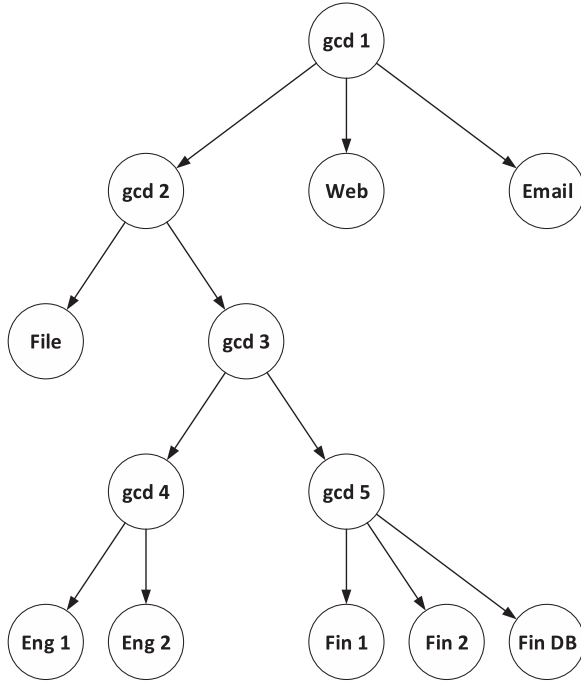



Fig. 15 – The network graph of the illustrative example after adding resources.

```

end for
end if
end while
end procedure

```

The robust network graph with resources attached for the illustrative example is shown in Fig. 15.

B5. ADD-1F-RESOURCES procedure

In this section, we present an algorithm for the ADD-1F-RESOURCES procedure.

```

procedure ADD-1F-RESOURCES(G, S)
  if S = ∅ then return error
  end if
  if r.p = 1_F then
    r.set ← r.set ∪ S; r.size ← r.size + |S|
    for each s ∈ S do
      s.is_resource ← true
      s.π ← {r}
      V ← V ∪ {s}
      E ← E ∪ {(r, s)}
    end for
  else
    Create node r'           ▷ Create new root r' of G
    r'.set ← r.set ∪ S; r'.size ← r.size + |S|
    r'.p ← 1_F; r'.weight ← weight of 1_F policy
    r'.π ← ∅
    V ← V ∪ {r'}             ▷ Add new root r' to G
    r.π ← {r'}               ▷ Attach r to r'
    E ← E ∪ {(r', r)}
    for each s ∈ S do
      s.is_resource ← true

```

```

s.π ← {r'}
V ← V ∪ {s}
E ← E ∪ {(r', s)}
end for
end if
end procedure

```

Appendix C. Auxiliary Functions for RNS Algorithm

In this section we present the auxiliary procedure ADD-NODE-TO-G and the function CREATE-NODE used by the RNS algorithm presented in Section 6.2.

The auxiliary ADD-NODE-TO-G procedure encapsulates the steps to add nodes (and any resources not already added to G in its set attribute) to G. The ADD-NODE-TO-G procedure always adds the node s to the root of r (if it exists). If s is a gcd node and has children that are resource nodes not added to G yet (indicated by exist = false), then s is added to G and all the resources in its s.set are created as new nodes and added to G as children of s. If s is a gcd node object, which is created to be a gcd node for existing nodes in G (indicated by exist = true), then s is added to G and all nodes in cset are added as its children (this includes removing their connection with r, and attaching them to s). In this case, it is possible for cset to contain a node having the same weight as that of s, in which case, we delete that node from G, and attach its children to s.

```

procedure ADD-NODE-TO-G(G, s, cset, exist)
  if G ≠ NULL then
    V ← V ∪ s; E ← E ∪ {(r, s)}
    s.π ← s.π ∪ {r}
  else
    if s.set = R then
      V ← V ∪ s; r ← s
    end if
  end if
  if s.set ≠ empty ∧ s ≠ r ∧ exist then ▷ Add gcd node with
    only resources not yet added to G as children
    for each res ∈ s.set do
      res = CREATE-NODE(res, ∅, false)
      V ← V ∪ res
      E ← E ∪ {(s, res)}
      res.π ← res.π ∪ {s}
    end for
    else if cset ≠ empty ∧ s ≠ r ∧ exist then ▷ Adding gcd
      node with children already added to G and possibly having
      gcd nodes as children.
      for each res ∈ cset do
        if res.isresource = false ∧ s.weight = res.weight
        then
          V ← V - {res}
          E ← E - {(r, res)}
          for each child c of res do
            E ← E - {(res, c)} ∪ {(s, c)}
            res.π ← res.π - {res} ∪ {s}
          end for
        else
          E ← E - {(r, res)} ∪ {(s, res)}
          res.π ← res.π - {r} ∪ {s}

```

```

    end if
  end for
end if
end procedure

```

The auxiliary CREATE-NODE function creates nodes so that they can be added to the graph G . While building G , two types of nodes, namely the resource nodes, and the GCD (firewall) nodes are created, and their attributes set according to the code given in the CREATE-NODE function. Since we mark nodes as temporary (nodes that are added to F and evaluated later) we introduce a new attribute *-temp-* which is added to all nodes. For permanent nodes *temp* is set to false, and for temporary nodes *temp* is set to true.

```

function CREATE-NODE( $s$ )
  Create node  $v$       ▷ Create node and initialize attribute
  values
  if  $|s| = 1 \wedge s \neq R$  then      ▷ Adding a resource
     $v.p \leftarrow$  policy of resource  $s$ 
     $v.weight \leftarrow$  weight of  $v.p$ 
     $v.set \leftarrow \emptyset$ ;  $v.size \leftarrow 0$ 
     $v.isresource \leftarrow true$ ;  $v.temp \leftarrow false$ 
  else
     $v.p \leftarrow$  GCD of all resource policies in  $s$ 
     $v.weight \leftarrow$  weight of  $v.p$ 
     $v.isresource \leftarrow false$ ;  $v.temp \leftarrow false$ 
    for each  $node \in s$  do
      if  $!node.isresource$  then
         $v.set \leftarrow v.set \cup node.set$ 
         $v.size \leftarrow v.size + node.size$ 
      else
         $v.set \leftarrow v.set \cup node$ 
         $v.size \leftarrow v.size + 1$ 
      end if
    end for
  end if
  return  $v$ 
end function

```

defined in Janicki and Khedri (2001), a tabular expression table is an organized sets of cells, where each cell contains a mathematical expression (Janicki and Khedri, 2001). This table contains a collection of headers and a grid indexed by these headers. A tabular expression representing a policy has two headers, H_1 which specifies the source IP numbers and protocols and H_2 which specifies the states and destination ports, and an internal grid G which specifies actions. Each grid cell represents a rule of the policy and specifies the ACTION state attribute value for its end state. For an example see Table D.2, which is the tabular expression representation of the Finance workstations and database policy (defined in Section 2).

We now outline the algorithm to compute the GCD of two policies represented as tabular expressions. For simplicity, we demonstrate it using an example. Consider the tabular expression representations of the Finance workstations and database policy Table D.2, and the Engineering workstations policy Table D.3 as the input to the gcd function. The algorithm first modifies these input tables to have matching headers. While creating the matching headers it is possible to have empty cells in the grids of the modified tables. These cells are filled with a value *false* which indicates that no rule exists corresponding to such cells in the policy. Since our input Tables D.2 and D.3 already have matching headers, we ignore this step. The algorithm then creates the GCD table layout by having the same headers as the matching headers of the (modified) input tables. After which, the value of each grid cell of the GCD table is computed starting with the first cell as follows:

1. If either of the corresponding input table cells have a value *false*, then its value is *false*.
2. Otherwise it is the union of their values. For example, if the corresponding input table cells have different values as seen in the grid cell [1, 5] of Table D.2 with value $a' = \text{ACCEPT}$, and Table D.3 with value $a' = \text{REJECT}$, then its value is the union of their values; that is, $a' = \text{ACCEPT} \vee a' = \text{REJECT}$.

The GCD of the Finance workstations and database policy and the Engineering workstations policy is given in Table D.4.

Appendix D. Tabular expressions and GCD computation

Tabular expressions are mathematical expressions in tabular form. They were formalized by Jin and Parnas (2010). As de-

Table D1 – Finance workstations and database policy represented as a tabular expression.

		$s \notin \{192.168.1.0/24, 192.168.2.0/24, 192.168.3.0/24, 192.168.4.0/24\}$									
		$s = 192.168.1.0/24$		$s = 192.168.2.0/24$		$s = 192.168.3.0/24$		$s = 192.168.4.0/24$			
		$p = \text{TCP}$	$p \neq \text{TCP}$	$p = \text{TCP}$	$p \neq \text{TCP}$	$p = \text{TCP}$	$p \neq \text{TCP}$	$p = \text{TCP}$	$p \neq \text{TCP}$	$p = \text{TCP}$	$p \neq \text{TCP}$
$st \in \{\text{RELATED}, \text{ESTABLISHED}\}$	$dport \in \{80, 25\}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$
	$dport \notin \{80, 25\}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$
$st = \text{INVALID}$	$dport \in \{80, 25\}$	$a' = \text{DROP}$	$a' = \text{DROP}$	$a' = \text{DROP}$	$a' = \text{DROP}$	$a' = \text{DROP}$	$a' = \text{DROP}$	$a' = \text{DROP}$	$a' = \text{DROP}$	$a' = \text{DROP}$	$a' = \text{DROP}$
	$dport \notin \{80, 25\}$	$a' = \text{DROP}$	$a' = \text{DROP}$	$a' = \text{DROP}$	$a' = \text{DROP}$	$a' = \text{DROP}$	$a' = \text{DROP}$	$a' = \text{DROP}$	$a' = \text{DROP}$	$a' = \text{DROP}$	$a' = \text{DROP}$
$st = \text{NEW}$	$dport \in \{80, 25\}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$	$a' = \text{REJECT}$	$a' = \text{REJECT}$	$a' = \text{REJECT}$	$a' = \text{REJECT}$	$a' = \text{REJECT}$	$a' = \text{REJECT}$	$a' = \text{DROP}$	$a' = \text{DROP}$
	$dport \notin \{80, 25\}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$	$a' = \text{REJECT}$	$a' = \text{REJECT}$	$a' = \text{REJECT}$	$a' = \text{REJECT}$	$a' = \text{REJECT}$	$a' = \text{REJECT}$	$a' = \text{DROP}$	$a' = \text{DROP}$

Table D2 – Engineering workstations policy represented as a tabular expression.

		$s \notin \{192.168.1.0/24, 192.168.2.0/24, 192.168.3.0/24, 192.168.4.0/24\}$									
			$s = 192.168.1.0/24$		$s = 192.168.2.0/24$		$s = 192.168.3.0/24$		$s = 192.168.4.0/24$		
		$p = \text{TCP}$	$p \neq \text{TCP}$	$p = \text{TCP}$	$p \neq \text{TCP}$	$p = \text{TCP}$	$p \neq \text{TCP}$	$p = \text{TCP}$	$p \neq \text{TCP}$	$p = \text{TCP}$	$p \neq \text{TCP}$
$st \in \{\text{RELATED}, \text{ESTABLISHED}\}$	$dport \in \{80, 25\}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$
	$dport \notin \{80, 25\}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$
$st = \text{INVALID}$	$dport \in \{80, 25\}$	$a' = \text{DROP}$	$a' = \text{DROP}$	$a' = \text{DROP}$	$a' = \text{DROP}$	$a' = \text{DROP}$	$a' = \text{DROP}$	$a' = \text{DROP}$	$a' = \text{DROP}$	$a' = \text{DROP}$	$a' = \text{DROP}$
	$dport \notin \{80, 25\}$	$a' = \text{DROP}$	$a' = \text{DROP}$	$a' = \text{DROP}$	$a' = \text{DROP}$	$a' = \text{DROP}$	$a' = \text{DROP}$	$a' = \text{DROP}$	$a' = \text{DROP}$	$a' = \text{DROP}$	$a' = \text{DROP}$
$st = \text{NEW}$	$dport \in \{80, 25\}$	$a' = \text{REJECT}$	$a' = \text{REJECT}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$	$a' = \text{REJECT}$	$a' = \text{REJECT}$	$a' = \text{REJECT}$	$a' = \text{REJECT}$	$a' = \text{DROP}$	$a' = \text{DROP}$
	$dport \notin \{80, 25\}$	$a' = \text{REJECT}$	$a' = \text{REJECT}$	$a' = \text{ACCEPT}$	$a' = \text{ACCEPT}$	$a' = \text{REJECT}$	$a' = \text{REJECT}$	$a' = \text{REJECT}$	$a' = \text{REJECT}$	$a' = \text{DROP}$	$a' = \text{DROP}$

Table D3 – Tabular expression representation of the GCD of Finance Workstations and Database policy, and Engineering Workstations policy.

		s = 192.168.1.0/24				s = 192.168.2.0/24				s = 192.168.3.0/24				s = 192.168.4.0/24				s ∈ {192.168.1.0/24, 192.168.2.0/24, 192.168.3.0/24, 192.168.4.0/24}
		p = TCP	p ≠ TCP	p = TCP	p ≠ TCP	p = TCP	p ≠ TCP	p = TCP	p ≠ TCP	p = TCP	p ≠ TCP	p = TCP	p ≠ TCP	p = TCP	p ≠ TCP	p = TCP	p ≠ TCP	
st ∈ {RELATED, ESTABLISHED}	dport ∈ {80, 25}	a' = ACCEPT	a' = ACCEPT	a' = ACCEPT	a' = ACCEPT	a' = ACCEPT	a' = ACCEPT	a' = ACCEPT	a' = ACCEPT	a' = ACCEPT	a' = ACCEPT	a' = ACCEPT	a' = ACCEPT	a' = ACCEPT	a' = ACCEPT	a' = ACCEPT	a' = ACCEPT	
	dport ∉ {80, 25}	a' = ACCEPT	a' = ACCEPT	a' = ACCEPT	a' = ACCEPT	a' = ACCEPT	a' = ACCEPT	a' = ACCEPT	a' = ACCEPT	a' = ACCEPT	a' = ACCEPT	a' = ACCEPT	a' = ACCEPT	a' = ACCEPT	a' = ACCEPT	a' = ACCEPT	a' = ACCEPT	
st = INVALID	dport ∈ {80, 25}	a' = DROP	a' = DROP	a' = DROP	a' = DROP	a' = DROP	a' = DROP	a' = DROP	a' = DROP	a' = DROP	a' = DROP	a' = DROP	a' = DROP	a' = DROP	a' = DROP	a' = DROP	a' = DROP	
	dport ∉ {80, 25}	a' = DROP	a' = DROP	a' = DROP	a' = DROP	a' = DROP	a' = DROP	a' = DROP	a' = DROP	a' = DROP	a' = DROP	a' = DROP	a' = DROP	a' = DROP	a' = DROP	a' = DROP	a' = DROP	
st = NEW	dport ∈ {80, 25}	a' = ACCEPT ∨ a' = REJECT	a' = ACCEPT ∨ a' = REJECT	a' = ACCEPT ∨ a' = REJECT	a' = ACCEPT ∨ a' = REJECT	a' = ACCEPT ∨ a' = REJECT	a' = REJECT	a' = REJECT	a' = REJECT	a' = REJECT	a' = REJECT	a' = REJECT	a' = REJECT	a' = DROP	a' = DROP	a' = DROP	a' = DROP	
	dport ∉ {80, 25}	a' = ACCEPT ∨ a' = REJECT	a' = ACCEPT ∨ a' = REJECT	a' = ACCEPT ∨ a' = REJECT	a' = ACCEPT ∨ a' = REJECT	a' = ACCEPT ∨ a' = REJECT	a' = REJECT	a' = REJECT	a' = REJECT	a' = REJECT	a' = REJECT	a' = REJECT	a' = REJECT	a' = DROP	a' = DROP	a' = DROP	a' = DROP	
		a' = REJECT	a' = REJECT	a' = REJECT	a' = REJECT	a' = REJECT	a' = REJECT	a' = REJECT	a' = REJECT	a' = REJECT	a' = REJECT	a' = REJECT	a' = REJECT	a' = DROP	a' = DROP	a' = DROP	a' = DROP	

Appendix E. Relative atomicity

```

1 -A INPUT -m state --state RELATED,ESTABLISHED -j ACCEPT
2 -A INPUT -m state --state INVALID -j DROP
3 -A INPUT -s 192.168.1.0/24 --sport 0:65535 -d 0.0.0.0/0 --
  dport 0:65535 -p all -j REJECT
4 -A INPUT -s 192.168.2.0/24 --sport 0:65535 -d 0.0.0.0/0 --
  dport 0:65535 -p all -j ACCEPT
5 -A INPUT -s 192.168.3.0/24 --sport 0:65535 -d 0.0.0.0/0 --
  dport 0:65535 -p all -j REJECT
6 -A INPUT -s 192.168.4.0/24 --sport 0:65535 -d 0.0.0.0/0 --
  dport 0:65535 -p all -j REJECT
7 -A INPUT -s 0.0.0.0/1 --sport 0:65535 -d 0.0.0.0/0 --dport
  0:79 -p udp -j DROP
8 -A INPUT -s 0.0.0.0/1 --sport 0:65535 -d 0.0.0.0/0 --dport
  0:79 -p icmp -j DROP
9 -A INPUT -s 0.0.0.0/1 --sport 0:65535 -d 0.0.0.0/0 --dport
  80:80 -p tcp -j DROP
10 -A INPUT -s 0.0.0.0/1 --sport 0:65535 -d 0.0.0.0/0 --dport
  81:65535 -p tcp -j DROP
11 -A INPUT -s 0.0.0.0/1 --sport 0:65535 -d 0.0.0.0/0 --dport
  80:80 -p udp -j DROP
12 -A INPUT -s 0.0.0.0/1 --sport 0:65535 -d 0.0.0.0/0 --dport
  81:65535 -p udp -j DROP
13 -A INPUT -s 0.0.0.0/1 --sport 0:65535 -d 0.0.0.0/0 --dport
  80:80 -p icmp -j DROP
14 -A INPUT -s 0.0.0.0/1 --sport 0:65535 -d 0.0.0.0/0 --dport
  81:65535 -p icmp -j DROP
15 -A INPUT -s 128.0.0.0/2 --sport 0:65535 -d 0.0.0.0/0 --dport
  0:79 -p udp -j DROP
16 -A INPUT -s 128.0.0.0/2 --sport 0:65535 -d 0.0.0.0/0 --dport
  0:79 -p icmp -j DROP
17 -A INPUT -s 128.0.0.0/2 --sport 0:65535 -d 0.0.0.0/0 --dport
  80:80 -p tcp -j DROP
18 -A INPUT -s 128.0.0.0/2 --sport 0:65535 -d 0.0.0.0/0 --dport
  81:65535 -p tcp -j DROP
19 -A INPUT -s 128.0.0.0/2 --sport 0:65535 -d 0.0.0.0/0 --dport
  80:80 -p udp -j DROP
20 -A INPUT -s 128.0.0.0/2 --sport 0:65535 -d 0.0.0.0/0 --dport
  81:65535 -p udp -j DROP
21 -A INPUT -s 128.0.0.0/2 --sport 0:65535 -d 0.0.0.0/0 --dport
  80:80 -p icmp -j DROP
22 -A INPUT -s 128.0.0.0/2 --sport 0:65535 -d 0.0.0.0/0 --dport
  81:65535 -p icmp -j DROP
23 -A INPUT -s 192.0.0.0/9 --sport 0:65535 -d 0.0.0.0/0 --dport
  0:79 -p udp -j DROP
24 -A INPUT -s 192.0.0.0/9 --sport 0:65535 -d 0.0.0.0/0 --dport
  0:79 -p icmp -j DROP
25 -A INPUT -s 192.0.0.0/9 --sport 0:65535 -d 0.0.0.0/0 --dport
  80:80 -p tcp -j DROP
26 -A INPUT -s 192.0.0.0/9 --sport 0:65535 -d 0.0.0.0/0 --dport
  81:65535 -p tcp -j DROP
27 -A INPUT -s 192.0.0.0/9 --sport 0:65535 -d 0.0.0.0/0 --dport
  80:80 -p udp -j DROP
28 -A INPUT -s 192.0.0.0/9 --sport 0:65535 -d 0.0.0.0/0 --dport
  81:65535 -p udp -j DROP
29 -A INPUT -s 192.0.0.0/9 --sport 0:65535 -d 0.0.0.0/0 --dport
  80:80 -p icmp -j DROP
30 -A INPUT -s 192.0.0.0/9 --sport 0:65535 -d 0.0.0.0/0 --dport
  81:65535 -p icmp -j DROP
31 -A INPUT -s 192.128.0.0/11 --sport 0:65535 -d 0.0.0.0/0 --
  dport 0:79 -p udp -j DROP
32 -A INPUT -s 192.128.0.0/11 --sport 0:65535 -d 0.0.0.0/0 --
  dport 0:79 -p icmp -j DROP
33 -A INPUT -s 192.128.0.0/11 --sport 0:65535 -d 0.0.0.0/0 --
  dport 80:80 -p tcp -j DROP
34 -A INPUT -s 192.128.0.0/11 --sport 0:65535 -d 0.0.0.0/0 --
  dport 81:65535 -p tcp -j DROP
35 -A INPUT -s 192.128.0.0/11 --sport 0:65535 -d 0.0.0.0/0 --
  dport 80:80 -p udp -j DROP
36 -A INPUT -s 192.128.0.0/11 --sport 0:65535 -d 0.0.0.0/0 --
  dport 81:65535 -p udp -j DROP
37 -A INPUT -s 192.128.0.0/11 --sport 0:65535 -d 0.0.0.0/0 --
  dport 80:80 -p icmp -j DROP
38 -A INPUT -s 192.128.0.0/11 --sport 0:65535 -d 0.0.0.0/0 --
  dport 81:65535 -p icmp -j DROP
39 -A INPUT -s 192.160.0.0/13 --sport 0:65535 -d 0.0.0.0/0 --
  dport 0:79 -p udp -j DROP
40 -A INPUT -s 192.160.0.0/13 --sport 0:65535 -d 0.0.0.0/0 --
  dport 0:79 -p icmp -j DROP
41 -A INPUT -s 192.160.0.0/13 --sport 0:65535 -d 0.0.0.0/0 --
  dport 80:80 -p tcp -j DROP
42 -A INPUT -s 192.160.0.0/13 --sport 0:65535 -d 0.0.0.0/0 --
  dport 81:65535 -p tcp -j DROP
43 -A INPUT -s 192.160.0.0/13 --sport 0:65535 -d 0.0.0.0/0 --
  dport 80:80 -p udp -j DROP
44 -A INPUT -s 192.160.0.0/13 --sport 0:65535 -d 0.0.0.0/0 --
  dport 81:65535 -p udp -j DROP
45 -A INPUT -s 192.160.0.0/13 --sport 0:65535 -d 0.0.0.0/0 --
  dport 80:80 -p icmp -j DROP
46 -A INPUT -s 192.160.0.0/13 --sport 0:65535 -d 0.0.0.0/0 --
  dport 81:65535 -p icmp -j DROP
47 -A INPUT -s 192.168.0.0/24 --sport 0:65535 -d 0.0.0.0/0 --
  dport 0:79 -p udp -j DROP
48 -A INPUT -s 192.168.0.0/24 --sport 0:65535 -d 0.0.0.0/0 --
  dport 0:79 -p icmp -j DROP
49 -A INPUT -s 192.168.0.0/24 --sport 0:65535 -d 0.0.0.0/0 --
  dport 80:80 -p tcp -j DROP
50 -A INPUT -s 192.168.0.0/24 --sport 0:65535 -d 0.0.0.0/0 --
  dport 81:65535 -p tcp -j DROP
51 ....

```

Fig. 16 – Snippet of Engineering workstations relative atomic policy.

Algorithm 1 Exponential Robust Network and Segmentation Algorithm.

```

1: procedure EXP-SEGMENTATION(R)
2:   S ← set of resources having  $1_F$  policy
3:   R ← R − S
4:   GCD ← COMPUTE-GCD(R)
5:   G ← BUILD-NETWORK-GRAPH(GCD, R)
6:   ADD-RESOURCES-TO-NET(G, GCD, R)
7:   OPTIMIZE-NETWORK-GRAPH(G)
8:   if  $S \neq \emptyset$  then
9:     ADD-1F-RESOURCES(G, S)
10:  end if
11: end procedure

```

CRedit authorship contribution statement

Neerja Mhaskar: Methodology, Validation, Formal analysis, Writing - original draft. Mohammed Alabbad: Methodology, Software, Validation, Formal analysis, Writing - original draft. Ridha Khedri: Conceptualization, Methodology, Formal analysis, Writing - original draft, Supervision, Funding acquisition.

REFERENCES

- Adão P, Focardi R, Guttman JD, Luccio FL. Localizing firewall security policies. In: 2016 IEEE 29th Computer Security Foundations Symposium (CSF); 2016. p. 194–209. doi:[10.1109/CSF.2016.21](https://doi.org/10.1109/CSF.2016.21).
- Al-Shabibi, A., McCauley, M., 2020. Pox controller. Available: <https://noxrepo.github.io/pox-doc/html> (Accessed: Mar 23, 2020).
- Alabbad M, Mhaskar N, Khedri R. Segmentation and architecture of networks with multiple entry points. *Computer Networks* 2020. Submitted
- Albanese M, Jajodia S, Noel S. Time-efficient and cost-effective network hardening using attack graphs. In: 42nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks; 2012. p. 1–12.
- Ammar M, Russello G, Crispo B. Internet of things: a survey on the security of IoT frameworks. *Journal of Information Security and Applications* 2018;38:8–27.
- Caprolu M, Raponi S, Di Pietro R. Fortress: an efficient and distributed firewall for stateful data plane sdn. *Security and Communication Networks* 2019;2019.
- Cheswick WR, Bellovin SM. *Firewalls and internet security: Repelling the wily hacker*. Addison-Wesley; 1994.
- Chowdhary A, Huang D, Alshamrani A, Sabur A, Kang M, Kim A, Velazquez A. Sdfw: sdn-based stateful distributed firewall. *arXiv preprint arXiv:1811.00634* 2018.
- Conti M, Dehghantanha A, Franke K, Watson S. Internet of things security and forensics: challenges and opportunities. *Future Generation Computer Systems* 2018;78:544–6.
- de Oliveira RLS, Schweitzer CM, Shinoda AA, Ligia Rodrigues Prete. Using mininet for emulation and prototyping software-defined networks. In: 2014 IEEE Colombian Conference on Communications and Computing (COLCOM); 2014. p. 1–6. doi:[10.1109/ColComCon.2014.6860404](https://doi.org/10.1109/ColComCon.2014.6860404).
- Desharnais J, Belkhter N, Sghaier SBM, Tchier F, Jaoua A, Mili A, Zaguia N. Embedding a demonic semilattice in a relation algebra. *Theor Comput Sci* 1995;149(2):333–60. doi:[10.1016/0304-3975\(94\)00271-J](https://doi.org/10.1016/0304-3975(94)00271-J).

- Dewri R, Poolsappasit N, Ray I, Whitley D. Optimal security hardening using multi-objective optimization on attack tree models of networks. In: *Proc. 14th ACM Conf. Comput. Commun. Security*; 2007. p. 204–13.
- Fægri TE, Hallsteinsen SO. A Software Product Line Reference Architecture for Security. In: *Software Product Lines - Research Issues in Engineering and Management*. Springer Berlin Heidelberg; 2006. p. 275–326. doi:10.1007/978-3-540-33253-4_8.
- Feamster N, Rexford J, Zegura E. The road to sdn: an intellectual history of programmable networks. *ACM SIGCOMM Computer Communication Review* 2014;44(2):87–98.
- Google Inc.. In: *Technical Report. Google's Approach to IT Security*. Google; 2012.
- Gries D, Schneider F. A logical approach to discrete math. New York: Springer-Verlag; 1993.
- Gupta V, Kaur S, Kaur K. Implementation of stateful firewall using pox controller. In: *2016 3rd International Conference on Computing for Sustainable Global Development (INDIACom)*. IEEE; 2016. p. 1093–6.
- Höfner P, Khedri R, Möller B. Feature algebra. In: Misra J, Nipkow T, Sekerinski E, editors. In: *FM 2006: Formal Methods*. 14th International Symposium on Formal Methods, McMaster University, Hamilton, Ontario, Canada: Springer; 2006. p. 300–15.
- Höfner P, Khedri R, Möller B. Algebraic view reconciliation. In: *6th IEEE International Conferences on Software Engineering and Formal Methods*. Cape Town, South Africa; 2008. p. 85–94.
- Höfner P, Khedri R, Möller B. An algebra of product families. *Software & Systems Modeling* 2011;10(2):161–82.
- Höfner P, Khedri R, Möller B. Supplementing product families with behaviour. *International Journal of Software and Informatics* 2011;245–66.
- Ingols K, Lippmann R, Piwowarski K. Practical attack graph generation for network defence. In: *Proc. Comput. Security Appl. Conf.*; 2006. p. 121–30.
- Janicki R, Khedri R. On a formal semantics of tabular expressions. *Sci Comput Program* 2001;39(1–2):189–213.
- Javid T, Riaz T, Rasheed A. A layer2 firewall for software defined network. In: *2014 Conference on Information Assurance and Cyber Security (CIACS)*. IEEE; 2014. p. 39–42.
- Jin Y, Parnas DL. Defining the meaning of tabular mathematical expressions. *Science of Computer Programming* 2010;75(11):980–1000. doi:10.1016/j.scico.2009.12.009. Special Section on the Programming Languages Track at the 23rd ACM Symposium on Applied Computing
- Kaur K, Kumar K, Singh J, Ghumman NS. Programmable firewall using software defined networking. In: *2015 2nd International Conference on Computing for Sustainable Global Development (INDIACom)*. IEEE; 2015. p. 2125–9.
- Khedri R, Jones O, Alabbad M. Defence in depth formulation and usage in dynamic access control. In: Maffei M, Ryan M, editors. In: *Principles of Security and Trust: 6th International Conference, POST 2017, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2017, Uppsala, Sweden, April 22–29, 2017, Proceedings*. Berlin, Heidelberg: Springer Berlin Heidelberg; 2017. p. 253–74. doi:10.1007/978-3-662-54455-6_12.
- Khedri R, Mhaskar N, MAlabbad. In: *Technical Report. On the Segmentation of Networks*. McMaster University; 2019.
- Kumar A, Srinath N. Implementing a firewall functionality for mesh networks using sdn controller. In: *2016 International Conference on Computation System and Information Technology for Sustainable Solutions (CSITSS)*. IEEE; 2016. p. 168–73.
- Lippmann R, Ingols K, Scott C, Piwowarski K, Kratkiewicz K, Artz M, Cunningham R. Validating and restoring defence in depth using attack graphs. In: *MILCOM 2006 - 2006 IEEE Military Communications conference*; 2006. p. 1–10.
- Madakam S, Ramaswamy R, Tripathi S. Internet of things (IoT): a literature review. *Journal of Computer and Communications* 2015;3(5):164–73. doi:10.4236/jcc.2015.35021.
- May CJ, Hammerstein J, Mattson J, Rush K. In: *Technical Report. Defence in Depth: Foundations for Secure and Resilient IT Enterprises*. Carnegie Mellon University; 2006.
- McKeown N, Anderson T, Balakrishnan H, Parulkar G, Peterson L, Rexford J, Shenker S, Turner J. Openflow: enabling innovation in campus networks. *ACM SIGCOMM Computer Communication Review* 2008;38(2):69–74.
- MJ Almoheri H, T Watson L, Yao D, Ou X. Security optimization of dynamic networks with probabilistic graph modeling and linear programming, 13; 2016. p. 474–87.
- Möller B, Struth G. wp Is wlp. In: *Lecture Notes in Computer Science*, 3929. Springer Berlin Heidelberg; 2006. p. 200–11.
- Morzhov S, Alekseev I, Nikitinskiy M. Firewall application for floodlight sdn controller. In: *2016 International Siberian Conference on Control and Communications (SIBCON)*. IEEE; 2016. p. 1–5.
- de Moura L, Bjørner N. Satisfiability modulo theories: An appetizer. In: Oliveira MVM, Woodcock J, editors. In: *Formal Methods: Foundations and Applications*. Berlin, Heidelberg: Springer Berlin Heidelberg; 2009. p. 23–36.
- National Security Agency (NSA). In: *Technical Report. Top 10 Information Assurance Mitigation Strategies*. NSA; 2013.
- Nunes BAA, Mendonca M, Nguyen X-N, Obraczka K, Turletti T. A survey of software-defined networking: past, present, and future of programmable networks. *IEEE Communications Surveys & Tutorials* 2014;16(3):1617–34.
- Parnas DL. In: *CRL Report 260. Tabular Representation of Relations*. Faculty of Engineering, McMaster University, Hamilton, Ontario, Canada: Communications Research Laboratory; 1992.
- Pena JGV, Yu WE. Development of a distributed firewall using software defined networking technology. In: *2014 4th IEEE International Conference on Information Science and Technology*. IEEE; 2014. p. 449–52.
- Peteanu, R., 2001. *Best Practices for Secure Development*. Poolsappasit N, Dewri R, Ray I. Dynamic security risk management using bayesian attack graphs, 9; 2012. p. 61–74.
- Rahman M, Al-Shaer E. A formal framework for network security design synthesis. In: *2013 IEEE 33rd International Conference on Distributed Computing Systems (ICDCS)*; 2013. p. 560–70.
- Rubel P, Ihde M, Harp S, Payne C. Generating policies for defence in depth. In: *21st Annual Computer Security Applications Conference (ACSAC'05)*; 2005. p. 10–514. doi:10.1109/CSAC.2005.26.
- RVacca J, Ellis S. *Firewalls jumpstart for network and systems administrators*. Elsevier; 2005.
- Scarfione K, Hoffman P. In: *Technical Report. Guidelines on Firewalls and Firewall Policy*. National Institute of Standards and Technology (NIST); 2009.
- Stawowski M. *The principles of network security design*. ISSA 2007.
- Stawowski M. *Network security architecture*. ISSA 2009.
- Suh M, Park SH, Lee B, Yang S. Building firewall over the software-defined network controller. In: *16th International Conference on Advanced Communication Technology*. IEEE; 2014. p. 744–8.
- Tran TV, Ahn H. A network topology-aware selectively distributed firewall control in sdn. In: *2015 International Conference on Information and Communication Technology Convergence (ICTC)*. IEEE; 2015. p. 89–94.
- Tran TV, Ahn H. Flowtracker: A sdn stateful firewall solution with adaptive connection tracking and minimized controller

- processing. In: *2016 International Conference on Software Networking (ICSN)*. IEEE; 2016. p. 1–5.
- U.S. Department of Homeland Security, 2016. Recommended Practice: Improving Industrial Control Systems Cybersecurity with Defense-in-Depth Strategies.
- Wagner N, Şahin C, Pena J, Riordan J, Neumayer S. Capturing the security effects of network segmentation via a continuous-time markov chain model. In: *Proceedings of the 50th Annual Simulation Symposium*. San Diego, CA, USA: Society for Computer Simulation International; 2017. p. 1–12.
- Wagner N, Şahin CŞ, Pena J, Streilein WW. Advances in Nature-Inspired Computing and Applications. In: Shandilya S NA, Shandilya S, editors. *Automatic Generation of Cyber Architectures Optimized for Security, Cost, and Mission Performance: A Nature-inspired Approach*. Springer, Cham; 2019. doi:[10.1007/978-3-319-96451-5_1](https://doi.org/10.1007/978-3-319-96451-5_1).
- Wagner N, Şahin CŞ, Winterrose M, Riordan J, Pena J, Hanson D, Streilein WW. Towards automated cyber decision support: A case study on network segmentation for security. In: *2016 IEEE Symposium Series on Computational Intelligence (SSCI)*; 2016. p. 1–10. doi:[10.1109/SSCI.2016.7849908](https://doi.org/10.1109/SSCI.2016.7849908).
- Wang L, Noel S, Jajodia S. Minimum-cost network hardening using attack graphs. *Comput. Commun.* 2006;29:3812–24.
- Dr. Neerja Mhaskar:** is an Assistant Professor at the Department of Computing and Software, McMaster University. She is a member of the Association for Computing Machinery (ACM). Her research

focuses on algorithms and data structures. In particular, she is interested in algorithms on strings for pattern matching, algorithms for designing a secure network, analyzing patterns in big data, developing tools and data structures for data compression, and information retrieval.

Mohammed Alabbad: is a Ph.D. student in the Department of Computing and Software, McMaster University, Hamilton, Ontario, Canada. He received his M.A.Sc. degree in Software Engineering from McMaster University in 2013, and his B.Sc. degree in Computer Science from King Saud University, Riyadh, Saudi Arabia, in 2006. His research interests include: feature modelling and network security.

Dr. Ridha Khedri: is a Professor of Software Engineering at the department of Computing and Software, McMaster University. He is a licensed professional engineer in the province of Ontario, and is a member of the Association for Computing Machinery and the IEEE Computer Society. He has authored over 80 peer-reviewed articles, and has supervised more than 20 graduate students. His research interests include algebraic methods in software engineering, information security policies analysis, cryptographic-key distribution scheme analysis, data cleansing, software product families, and formal software requirements analysis, and medical device software.